CSC1046 – Object-Oriented Analysis & Design

MyReversi

Phase 2: Product Design & Class Design

Assignment 2 – Group 13

Team members	
Anna Langan	20401
Ailbhe de Burca	38069
Timothy Loomes	20540
Diane Gerard	133485
Blanche Puiseux	133479
Michel Melhem	132978

Module	CSC1046, Dublin City University
Deliverable	Phase 2 – Product Design and Class Design
Due	Friday 23 October 2026, 17:00 (end of Week 7)
Source code	github.com/MichelMelhem/MyReversiGrp13
Diagram sources	`docs/phase2/report/diagrams/` (PlantUML)

Contents

1 Introduction **2**

2 Revised Use Cases **3**

3 Refined Class Diagrams **7**

3.1 What changed since Phase 1 10

3.2 Association details 11

3.3 Open questions from the Phase 1 walkthrough, now resolved 12

4 Object Diagrams **12**

5 User Interface Mock-ups **14**

6 State Machines **16**

7 Sequence Diagrams **20**

8 Communication Diagrams **23**

9 Class Skeletons **26**

9.1 Game rules – `domain.py` 26

9.2 Server – `server.py` 28

9.3 Browser client – `client.js` 29

10 Technical Vulnerability Review **30**

11 Design Consistency Review **31**

12 Team Meeting Records **32**

12.1 Meeting 1 – Wednesday 7 October 2026, 12:00 33

12.2 Meeting 2 – Wednesday 14 October 2026, 12:00 33

12.3 Meeting 3 – Wednesday 21 October 2026, 12:00 33

12.4 Task board (status on 22 October 2026) 34

12.5 Activity log (from GitHub commits and Google Doc history) 34

13 Use of AI **34**

A Diagram Source Files **35**

1 Introduction

MyReversi is an Internet version of Reversi (Othello) for two players, or one player against the computer. In Phase 1 we wrote the requirements, scenarios, use cases and a first domain class diagram. This document is the Phase 2 deliverable: it turns that analysis into a design that is detailed enough to code from.

Architecture in one paragraph. The server (Python, Django REST framework) is the only authority on the rules. The browser client (HTML/CSS and JavaScript) shows the board and sends the square the player clicked. Clients ask the server for the latest state every 1.5 seconds (polling over HTTP/JSON). Each seat in a game has a secret token, so a player can only move for their own colour. Games and moves are saved in a database so that both browsers see the same board, and so that history, replay and the leaderboard work.

Contents of this deliverable

#	Required item	Section
1	Revised use cases and use case diagram	2
2	Refined class diagrams (plus object diagrams)	3, 4
3	User interface mock-ups	5
4	State machines (two state diagrams and two sub-state diagrams, plus two extra)	6
5	Sequence diagrams (two diagrams and two sub-sequence diagrams, UML 2.x)	7
6	Communication diagrams (four, one for each sequence diagram)	8
7	Class skeletons in our implementation languages	9
8	Minutes of team meetings (date, attendance, actions)	12
9	PlantUML text files	Appendix A
–	Technical vulnerability review (Phase 1 feedback)	10
–	Design consistency review	11
–	Use of AI	13

Response to the Phase 1 feedback

Feedback	What we did in Phase 2
Problems in the use cases	Wrong cross-references fixed, open questions answered and written into the use cases (Section 2).
Formal review of technical vulnerabilities	STRIDE review of 15 risks, each linked to a class and operation (Section 10).
Clearer Agile progress tracking	Meeting records now include a task board with owner, status and date, and a dated activity log taken from GitHub and the Google Doc history (Section 12).

Who did what in Phase 2

Work was split at the meeting of 7 October along the lines of each person's Phase 1 work.

Member	Phase 2 responsibility
Timothy Loomes	Revised use cases and use case diagram; sequence diagrams SD-1 and SD-1a
Ailbhe de Burca	Sequence diagrams (with Tim); review of the scenarios
Blanche Puiseux	Object diagrams; communication diagrams
Anna Langan	State machine diagrams; structured walkthrough
Diane Gerard	User interface mock-ups; client navigation state machine; meeting notes
Michel Melhem	Refined class diagrams; class skeletons; vulnerability review; Git integration and final report

2 Revised Use Cases

The Phase 1 use cases had a few errors, and the walkthrough left several open questions. Table 1 lists what changed. The revised use cases follow; the use case diagram is in Figure 1.

Table 1: Changes to the Phase 1 use cases

Use case	Change
UC-01	Precondition referred to UC-01 itself; removed. Colours are now drawn at random. New variation: players are only paired with someone who chose the same board size. Invite by username kept as a planned extension (it appears in the mock-ups).
UC-02	Precondition now refers to UC-01 (it wrongly said UC-02). Pause (“*b”) removed: there is no pause in this version. Disconnection now has a clear outcome (forfeit after 30 s). New variation: legal moves are highlighted.
UC-04	Rematch question answered (no rematch, players return to the lobby). Forfeit after a 30-second reconnection window.
UC-05	Step 4 pointed to “UC-05 End Game”; corrected to UC-04. Difficulty is in scope: easy, normal or hard.
Diagram	Typo “Agianst” fixed; UC-05 now shows that it includes UC-02 (Make Move).

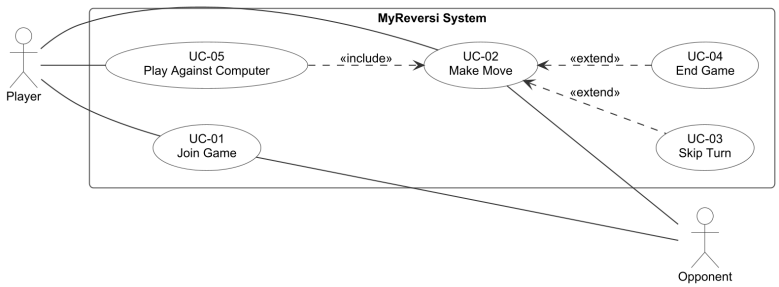


Figure 1: Revised use case diagram

UC-01 Join Game	
Goal	Player is paired with another online player and a new game begins; both players see the starting board.
Scope & level	MyReversi system, user goal
Actors	Primary: Player. Supporting: Opponent
Precondition	Player is connected and in the game lobby.
Trigger	Player chooses to play online from the lobby.
Main scenario	1. System finds another player waiting for a game with the same board size. 2. System pairs the two players and draws Black and White at random. 3. System shows both players the starting board: the chosen grid (8×8 by default) with two black and two white pieces on the four centre squares. 4. System tells each player their colour and their opponent's name. 5. System gives the first turn to Black (UC-02).
Extensions	1a. No other player is waiting: the player is told they are waiting for an opponent. When another player joins, resume at step 2. 1b. Player cancels while waiting: back to the lobby. EXIT (no game started). 2a. The waiting player has been gone for more than 30 s: their game is cancelled and the search continues. Resume at step 1.
Variations	1′. Board size filter: players are only paired if they chose the same size (6×6, 8×8 or 10×10). 1″. Planned extension: invite a specific opponent by username (see mock-ups, Figure 7).
Success guarantee	Both players see the same starting board, know their colour and opponent, and Black is to move.
Minimal guarantee	No game started; the player is back in the lobby and knows why. A player is never in two games at once.
Special requirements	Once two players are waiting, they see the starting board within 2 seconds.
Notes	Decision: colours are assigned at random (was an open question in Phase 1).

UC-02 Make Move	
Goal	Player places a piece on a legal square, every outflanked opponent piece is flipped and the turn passes to the opponent.
Scope & level	MyReversi system, user goal
Actors	Primary: Player whose turn it is. Supporting: Opponent
Preconditions	1. A game is in progress (UC-01 or UC-05). 2. It is the player's turn and they have at least one legal move (otherwise UC-03).
Trigger	The turn passes to the player.
Main scenario	1. System shows the player it is their turn and highlights the legal squares. 2. Player selects a square (e.g. Black selects D3). 3. System places the piece and flips every opponent piece outflanked in any of the eight directions. 4. System shows the updated board and the piece count of each colour to both players. 5. System passes the turn to the opponent and tells both players whose turn it is.
Extensions	2a. The square is not a legal move (occupied, or outflanks nothing): the player is told the move is not allowed; the board is unchanged. Resume at step 1. 5a. The opponent has no legal move but the player does: UC-03 Skip Turn (extend). 5b. Neither player has a legal move (e.g. full board): UC-04 End Game (extend). EXIT. *a. A player loses connection: if they do not come back within 30 s they forfeit (UC-04, variation 3′).
Variations	2′. The player clicks a highlighted square on the board.
Success guarantee	The piece is placed, all and only the outflanked pieces are flipped, both players see the same board and the turn has passed correctly.
Minimal guarantee	The board only changes through a legal move; both players always see the same board.
Special requirements	The new board appears on both screens within about 1.5 seconds (one polling interval).

UC-03 Skip Turn	
Goal	When the next player has no legal move, both players are told and the turn goes back to the player who just moved.
Scope & level	Subfunction (extends UC-02 at step 5)
Actors	Primary: Player whose turn is skipped. Supporting: Opponent
Preconditions	A move has just been made (UC-02); the next player has no legal move but the player who just moved does.
Trigger	System finds that the next player has no legal move.
Main scenario	1. System tells both players that the next player must pass (e.g. "White has no legal move – Black plays again"). 2. System gives the turn back to the player who just moved. 3. Game continues with UC-02.
Extensions	None: if neither player can move, UC-04 applies instead.
Variations	1′. Against the computer (UC-05), only the human player is told.
Success guarantee	Both players know the turn was skipped and the right player is to move.
Minimal guarantee	A player is never skipped while they still have a legal move.

UC-04 End Game	
Goal	When neither player can move, the game stops and both players see the same result.
Scope & level	Subfunction (extends UC-02 at step 5)
Actors	Primary: Player. Supporting: Opponent
Precondition	A game is in progress.
Trigger	After a move, neither player has a legal move.
Main scenario	1. System stops accepting moves. 2. System counts the pieces of each colour. 3. System shows both players the final board and score (e.g. Black 38 – White 26) and declares the player with more pieces the winner. 4. System updates the leaderboard and offers to return to the lobby.
Extensions	3a. Same number of pieces: the game is a draw. Resume at step 4.
Variations	3′. A player leaves, or is disconnected for more than 30 s: the other player wins by forfeit whatever the piece count.
Success guarantee	Both players see the same final board and result; no more moves can be made.
Minimal guarantee	The result shown to both players is identical.
Notes	Decision: no rematch; players go back to the lobby.

UC-05 Play Against Computer	
Goal	The player plays a complete game against a computer opponent.
Scope & level	MyReversi system, user goal
Actors	Primary: Player. Supporting: none (the computer is part of the system)
Precondition	The player is connected and in the lobby.
Trigger	The player chooses to play against the computer.
Main scenario	1. System starts a game with the player as Black and the computer as White and shows the starting board. 2. The player makes a move (UC-02). 3. System chooses the computer's move, places it, flips the outflanked pieces and shows the new board. 4. Steps 2–3 repeat until neither side can move, then UC-04.
Extensions	2a. The player has no legal move: UC-03, then step 3. 3a. The computer has no legal move: UC-03, then step 2. *a. The player quits: system asks for confirmation and returns to the lobby. EXIT (game abandoned).
Variations	1′. The player chooses a difficulty: easy, normal or hard.
Success guarantee	A full game is played and the result shown (UC-04).
Minimal guarantee	The game is abandoned and the player is back in the lobby.
Special requirements	The computer moves within 2 seconds, with a short pause so the player can see what changed.

Glossary	
Board	Square grid of 6×6, 8×8 (default) or 10×10. Columns are letters (A, B, . . .), rows are numbers (1, 2, . . .).
Piece	A disc that is black on one side and white on the other.
Outflank	Trap a straight line of opponent pieces between the new piece and another piece of the player's colour.
Flip	Turn an outflanked piece over to the player's colour.
Legal move	Placing a piece on an empty square so that it outflanks at least one opponent piece.
Skip (pass)	Giving up a turn because no legal move exists.
Lobby	Where a player enters their name and chooses an online or computer game.
Forfeit	Losing a game by leaving it or by being disconnected for more than 30 seconds.

3 Refined Class Diagrams

The Phase 1 domain class diagram (Blanche and Michel) has been refined into three diagrams:

- Figure 2: the domain classes (package `game`) with all attributes, operations and associations;
- Figure 3: the client and server classes that replace `Server` and `Client` from Phase 1;
- Figure 4: where each part runs (section 3.2.1 of the brief).

These class and operation names are used in every other diagram of this document.

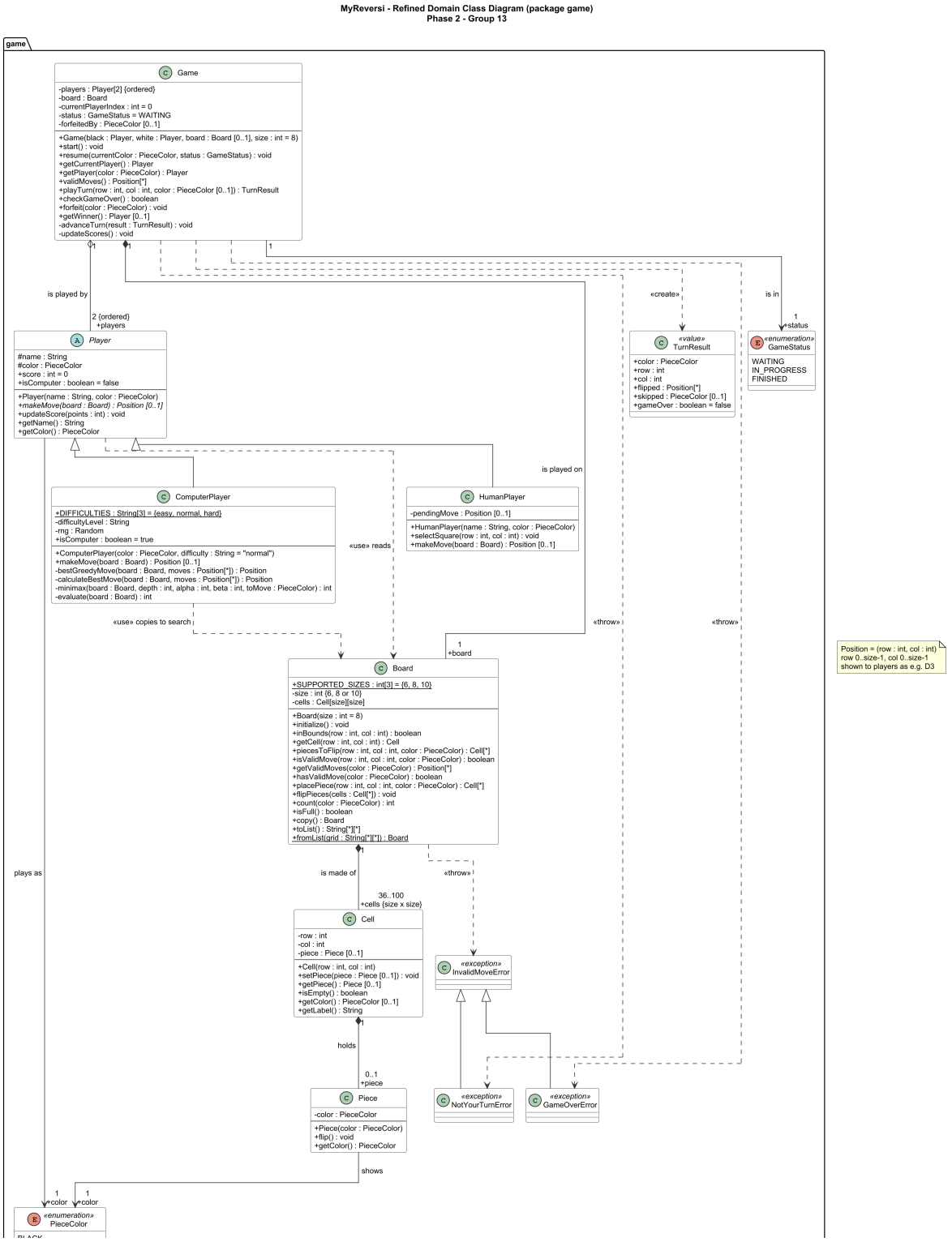


Figure 2: Refined domain class diagram (package `game`)

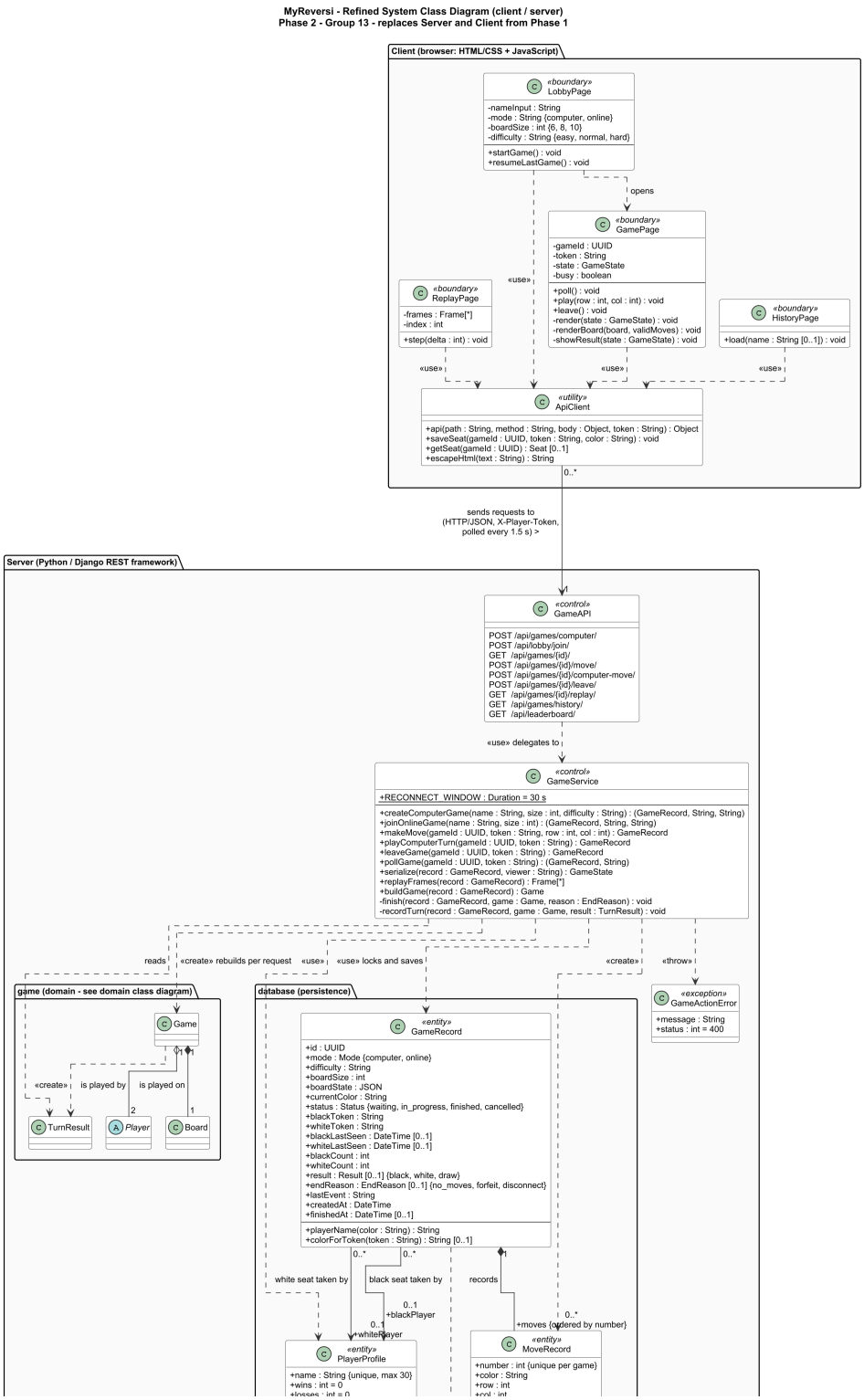


Figure 3: Refined system class diagram (client and server)

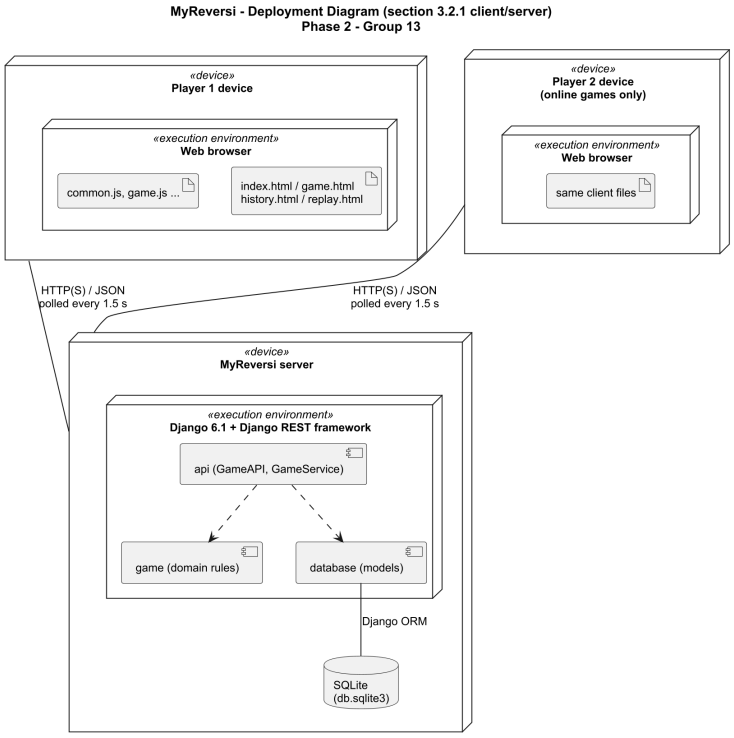


Figure 4: Deployment diagram

3.1 What changed since Phase 1

#	Phase 1	Phase 2	Why
1	Server (sockets, broadcastGameState)	GameAPI (HTTP endpoints) + GameService	We use Django over HTTP with clients polling every 1.5 s, not raw sockets. The API classes stay thin.
2	Client (connectToServer, sendMove, updateUI)	ApiClient + LobbyPage, GamePage, HistoryPage, ReplayPage	One boundary class per screen; ApiClient is the only class that talks to the server.
3	Server 1 o– * Client	ApiClient 0..* –> 1 GameAPI	The server no longer holds client objects; a client is identified by a secret token per game.
4	Client 1 *– Game	removed	The client never owns a Game. It only shows the state the server sends, so the rules cannot be bypassed.
5	Game.currentTurn, gameState : String	currentPlayerIndex : int, status : GameStatus	An enumeration only allows valid states. The state machines use the same names.
6	initializeGame, switchTurn, checkGameOver	start, resume, playTurn → TurnResult, validMoves, forfeit, getWinner, checkGameOver	Each use case has an operation: UC-01 start, UC-02 playTurn, UC-03 advanceTurn, UC-04 getWinner / forfeit.
7	Move	TurnResult and MoveRecord	A move is just (row, col) in; what we need out is what happened (flips, skip, game over) and a saved copy for history and replay.
8	Board.rows, Board.cols	Board.size ∈ {6, 8, 10}	Custom board sizes; Reversi boards are always square.

#	Phase 1	Phase 2	Why
9	isValidMove, executeFlip	piecesToFlip, isValidMove, getValidMoves, hasValidMove, placePiece, flipPieces, count, copy	getValidMoves highlights legal moves; hasValidMove drives skip (UC-03) and end (UC-04); copy is used by the computer's search.
10	Cell.isObstacle	removed	Obstacle tiles were dropped; custom boards are done by size instead.
11	Cell 0..1 o– 1 Piece	Cell 1 *– 0..1 Piece	The multiplicities were on the wrong ends. A cell holds zero or one piece.
12	Board 1 *– * Cell	Board 1 *– 36..100 Cell	The exact number is known from the supported sizes.
13	Piece.color : String	Piece.color : PieceColor	No invalid colours; flip() sets color = color.opponent().
14	difficultyLevel : String, calculateHeuristicMove()	difficultyLevel ∈ {easy, normal, hard}, makeMove, minimax, evaluate	Answers "is difficulty in scope?" (yes).
15	–	PlayerProfile, GameRecord, MoveRecord	Needed for move record, game history and leaderboard, and so that both browsers share one saved game.
16	–	InvalidMoveError, NotYourTurnError, GameOverError, GameActionError	Named errors give the "move not allowed" feedback of UC-02 extension 2a.

3.2 Association details

Read each name in the arrow direction (e.g. "Game is played on Board").

Association	Type	Multiplicity	Role	Meaning
Game – Board: is played on	Composition	1 – 1	board	The board is created with the game and dies with it.
Game – Player: is played by	Aggregation	1 – 2 {ordered}	players	Index 0 is Black, 1 is White. Players are created outside the game.
Board – Cell: is made of	Composition	1 – 36..100	cells	6×6, 8×8 or 10×10 grid.
Cell – Piece: holds	Composition	1 – 0..1	piece	A cell is empty or holds one piece.
Game – GameStatus: is in	Association	1 – 1	status	Current state of the game.
Piece – PieceColor: shows	Association	* – 1	color	Side facing up.
Player – PieceColor: plays as	Association	* – 1	color	Colour given at the start.
Player ◁ HumanPlayer, ComputerPlayer	Generalisation	–	–	Both implement makeMove(board).
InvalidMoveError ◁ NotYourTurnError, GameOverError	Generalisation	–	–	All rule errors can be caught as one type.
Game → TurnResult	Dependency ≪create≫	–	–	playTurn returns a new TurnResult.
ApiClient – GameAPI: sends requests to	Association	0..* – 1	–	HTTP/JSON, token in the X-Player-Token header.

Association	Type	Multiplicity	Role	Meaning
Pages → ApiClient	Dependency ≪use≫	–	–	Only ApiClient makes network calls.
GameAPI → GameService	Dependency ≪use≫	–	–	Views delegate all work.
GameService → Game	Dependency ≪create≫	–	–	buildGame rebuilds Game, Board and Players from a GameRecord on each request.
GameRecord – PlayerProfile: black / white seat taken by	Association	0..* – 0..1	blackPlayer, whitePlayer	Null means the computer, or nobody yet.
GameRecord – MoveRecord: records	Composition	1 – 0..* {ordered}	moves	Deleting a game deletes its moves.

3.3 Open questions from the Phase 1 walkthrough, now resolved

Question	Decision	Where in the design
Colours random or by joining order?	Random once two players are paired	GameService.joinOnlineGame
Rematch after a game?	No; players go back to the lobby	GamePage.showResult
Pauses?	No pause; leaving a game is a forfeit	GameService.leaveGame, Game.forfeit
Is computer difficulty in scope?	Yes: easy, normal, hard	ComputerPlayer.DIFFICULTIES
Does a disconnected player forfeit?	Yes, after 30 seconds	GameService.RECONNECT_WINDOW

4 Object Diagrams

The object diagrams (Blanche) show snapshots of the classes in Figure 2 at two moments of a game. OD-1 is the start of a game against the computer; OD-2 is an online game in progress and also shows the saved server objects behind it.

OD-1 Start of a game against the computer (8 x 8, after Game.start())

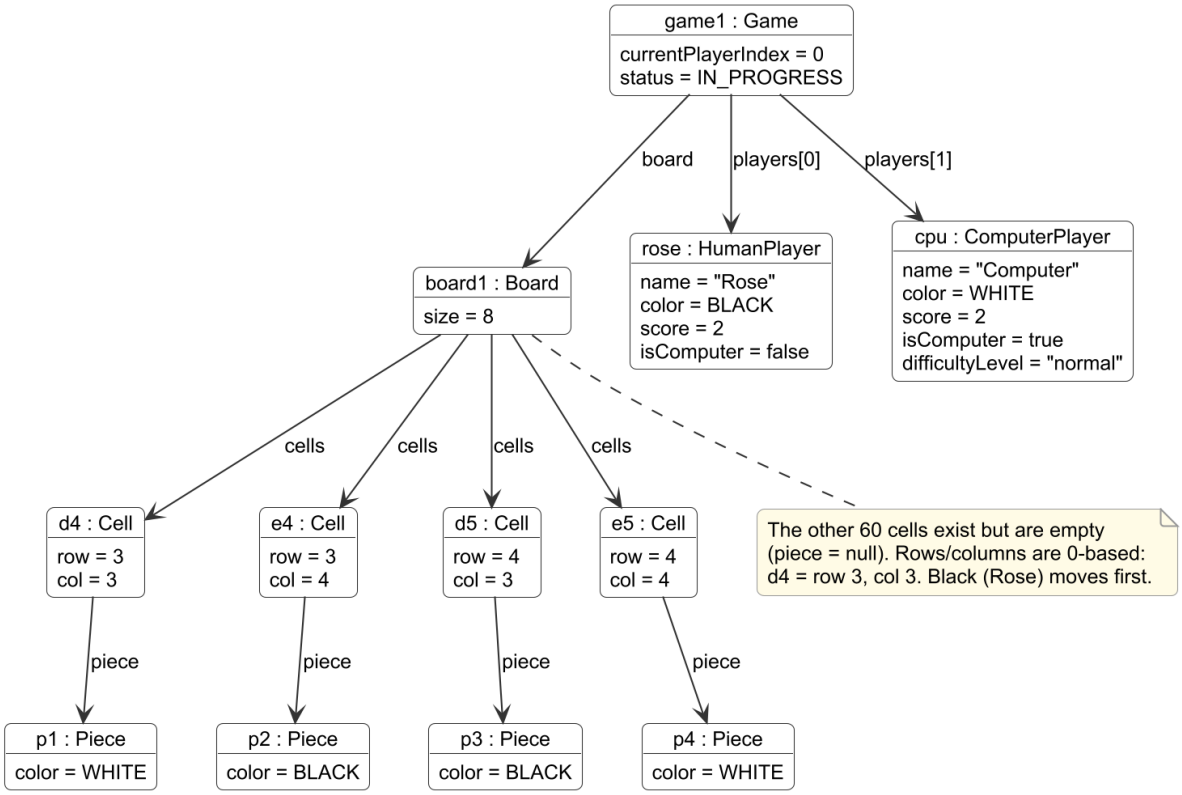


Figure 5: OD-1: start of a game against the computer

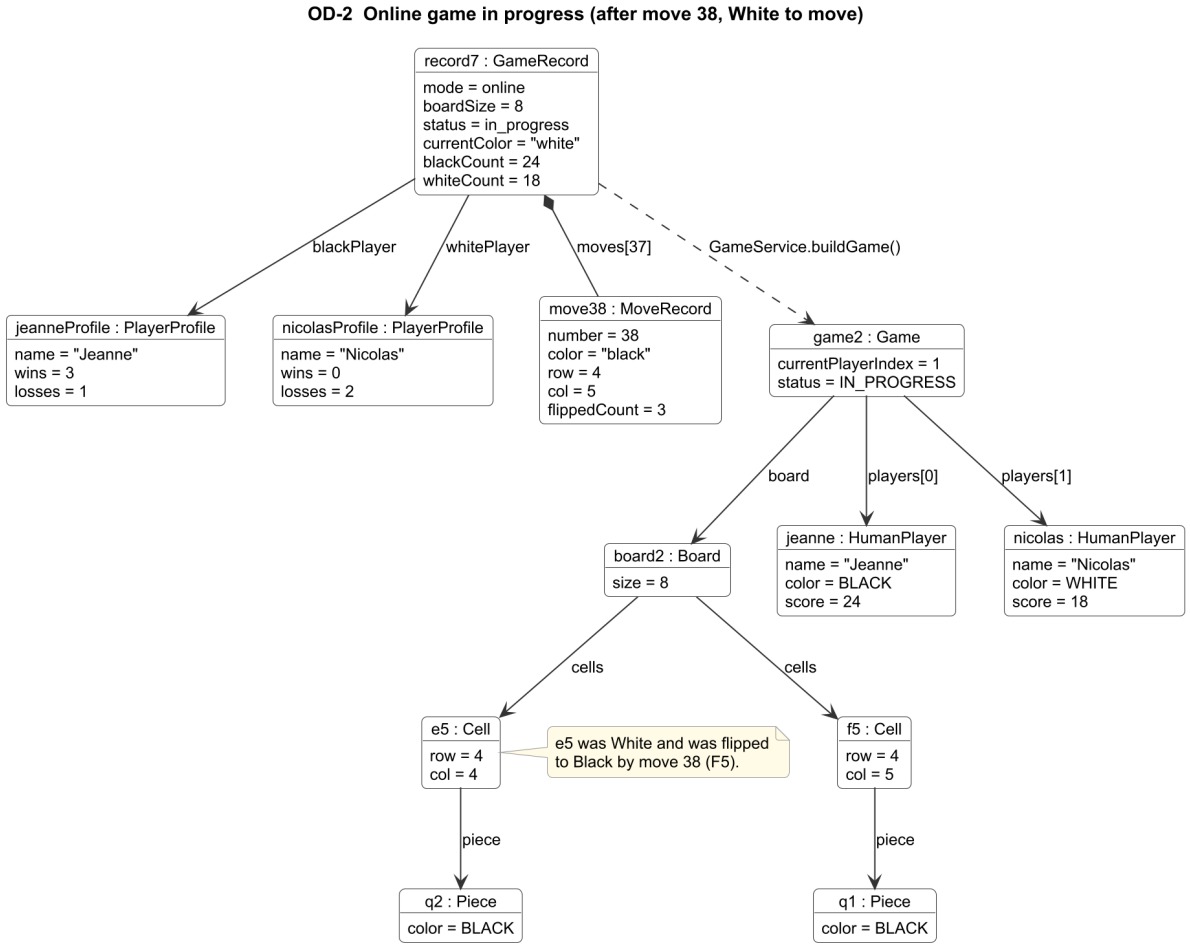
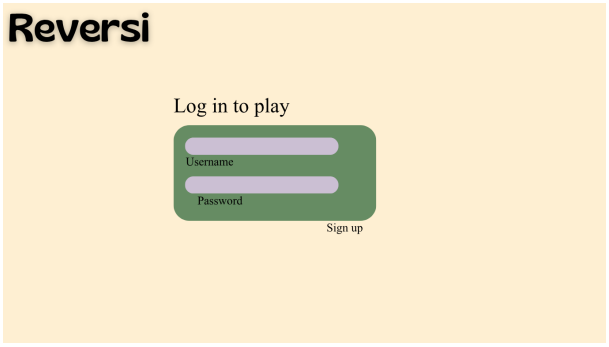


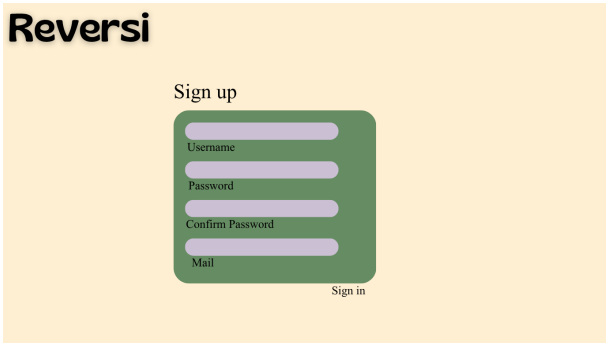
Figure 6: OD-2: online game in progress, after move 38

5 User Interface Mock-ups

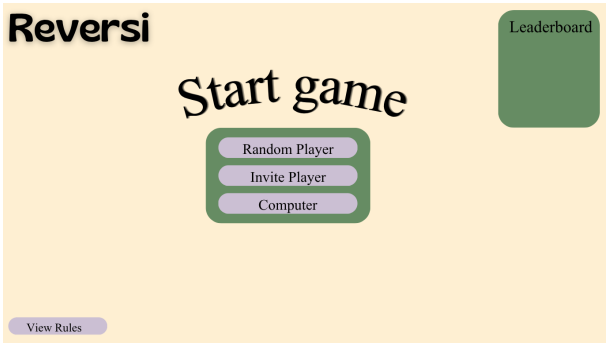
The mock-ups (Diane, made in Canva) show how the game should look. The lobby, game screen and legal-move highlighting map directly onto `LobbyPage` and `GamePage`. The sign-in, sign-up and invitation screens are planned extensions: in the current design a player is identified by the name typed in the lobby (see V9 in Section 10). Their navigation is modelled in SM-6 (Figure 13).



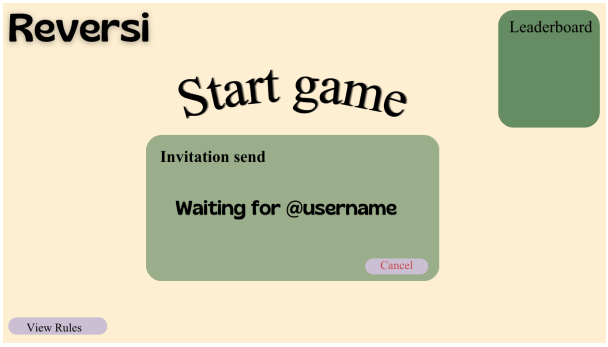
(a) Sign in (planned)



(b) Sign up (planned)



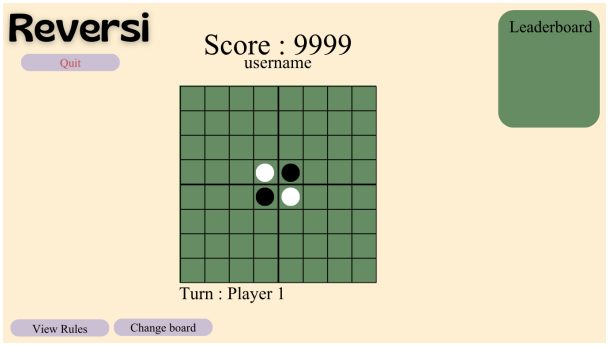
(c) Lobby – LobbyPage



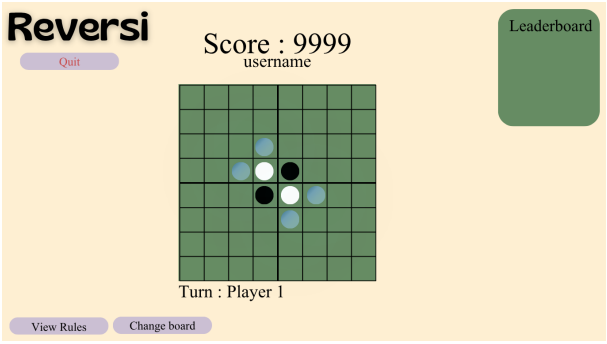
(d) Invite a player (planned)



(e) Invitation received (planned)



(f) Game board – GamePage



(g) Legal moves highlighted (UC-02, variation 1′)

Figure 7: User interface mock-ups

Screen	Elements	Design classes / states
Lobby	Random player, invite player, computer; leaderboard; rules	LobbyPage.startGame(); SM-3 Lobby
Game board	Board, score, whose turn, quit, change board, leaderboard	GamePage.render(), play(), leave(); SM-4
Highlighted moves	Light-blue discs on legal squares	Board.getValidMoves(); UC-02 step 1
Sign in / up, invite	Account and invitation screens	Planned extension; SM-6 grey states

6 State Machines

Following section 3.2.1 of the brief, the behaviour is shown as a hierarchy of state machines. The four required diagrams are two state diagrams and two sub-state diagrams; every transition is named and has a guard where a choice is made:

Level	Server side (Game)	Client side (GamePage)
State diagram	SM-1 Game (GameStatus)	SM-3 GamePage
Sub-state diagram	SM-2 IN_PROGRESS	SM-4 Playing
Extra	SM-5 Cell	SM-6 Client navigation (target UI)

State names match `GameStatus` (WAITING, IN_PROGRESS, FINISHED) and the server status `cancelled`.

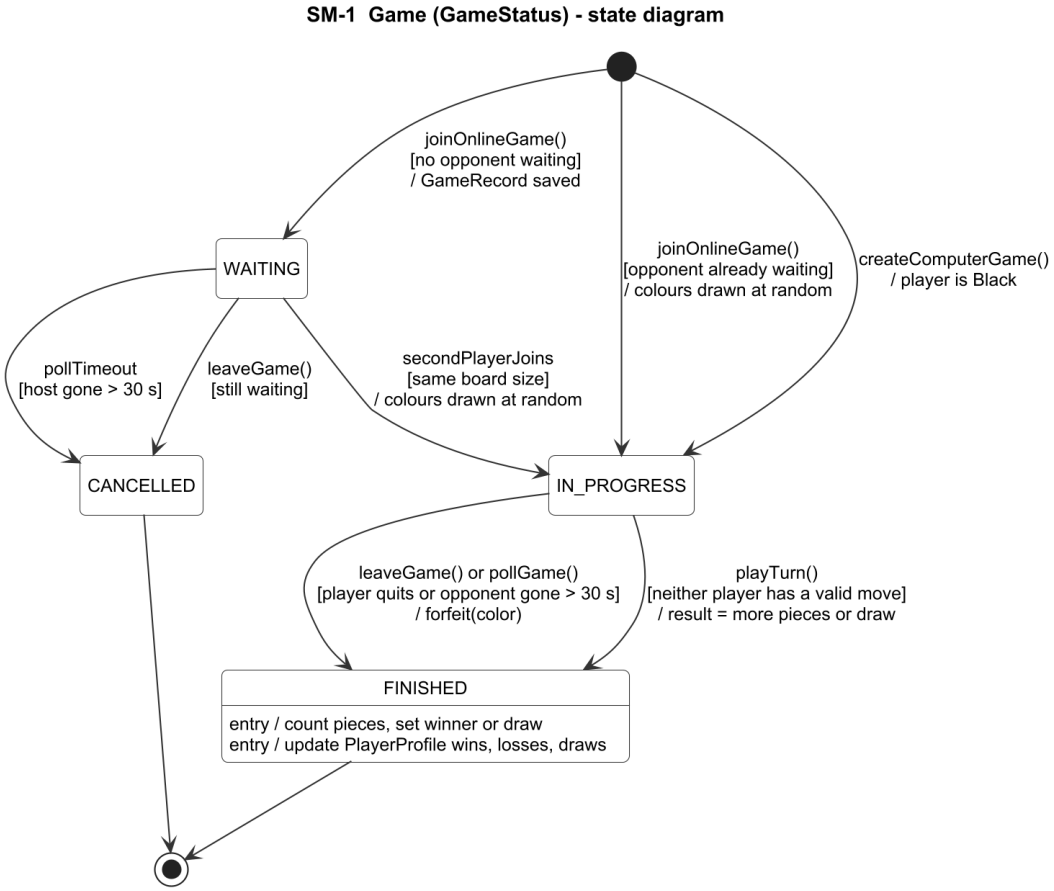


Figure 8: SM-1: Game – state diagram

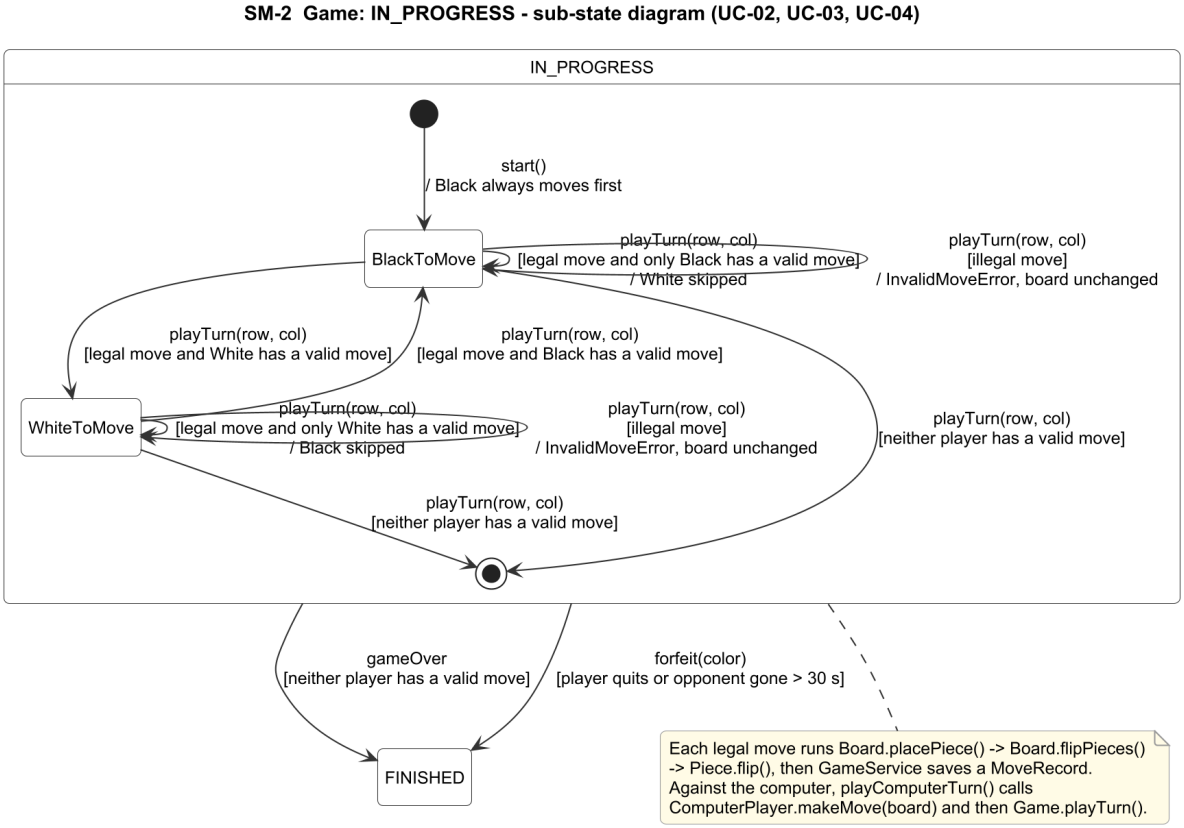


Figure 9: SM-2: Game IN_PROGRESS – sub-state diagram

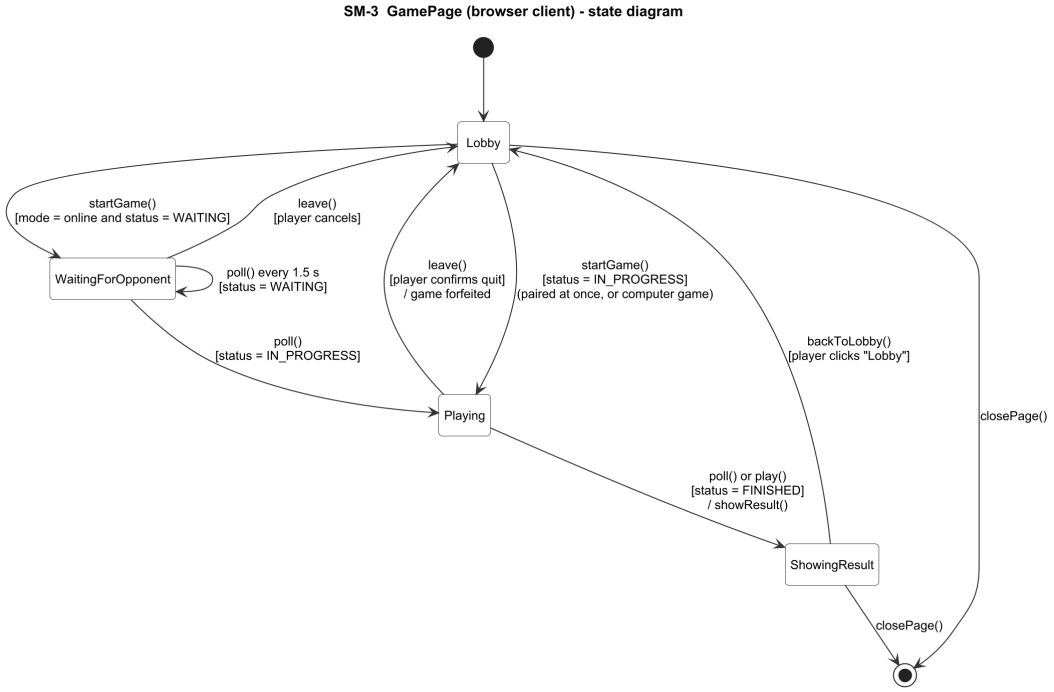


Figure 10: SM-3: GamePage – state diagram

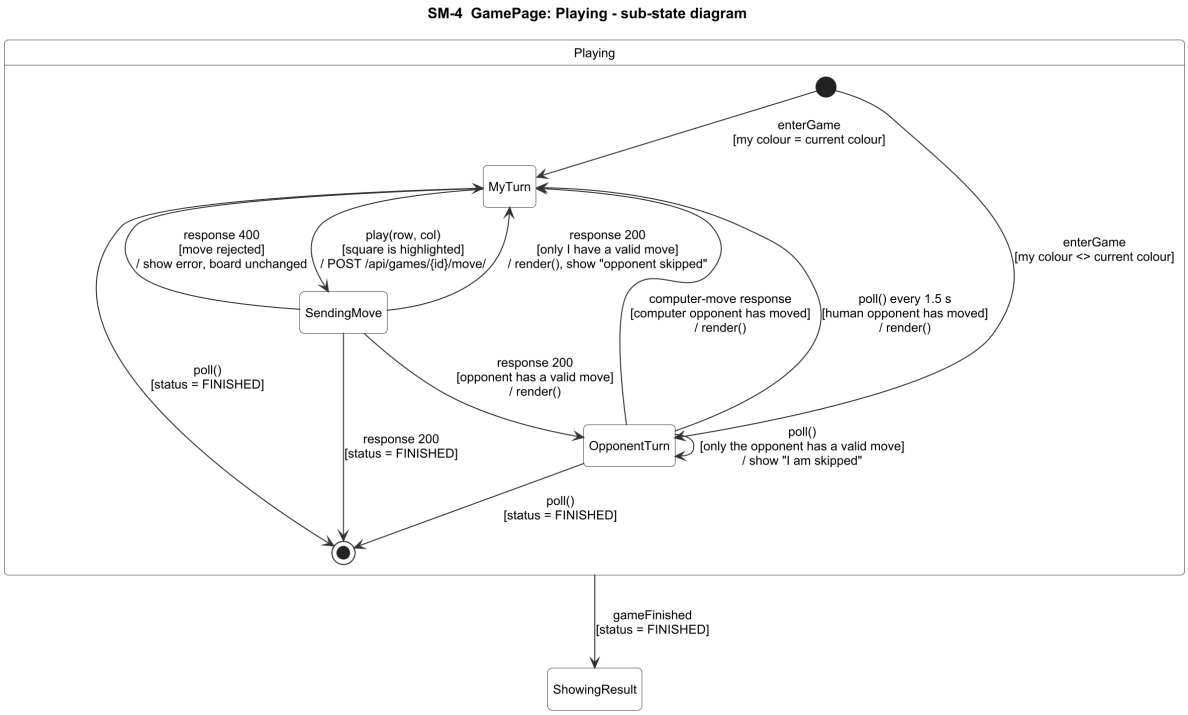


Figure 11: SM-4: GamePage Playing – sub-state diagram

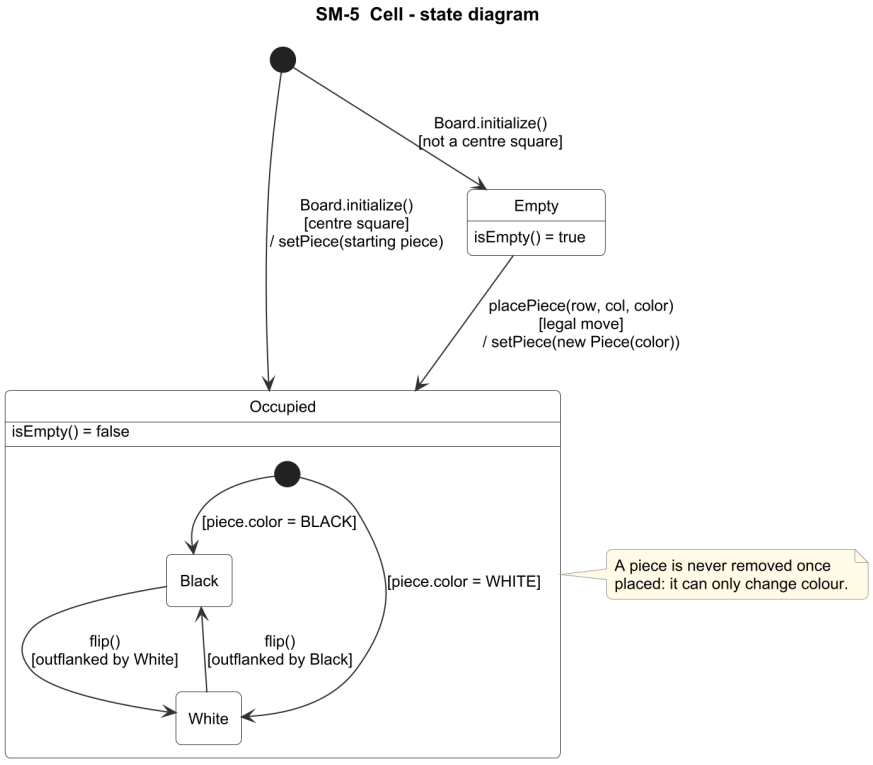


Figure 12: SM-5: Cell – state diagram (extra)

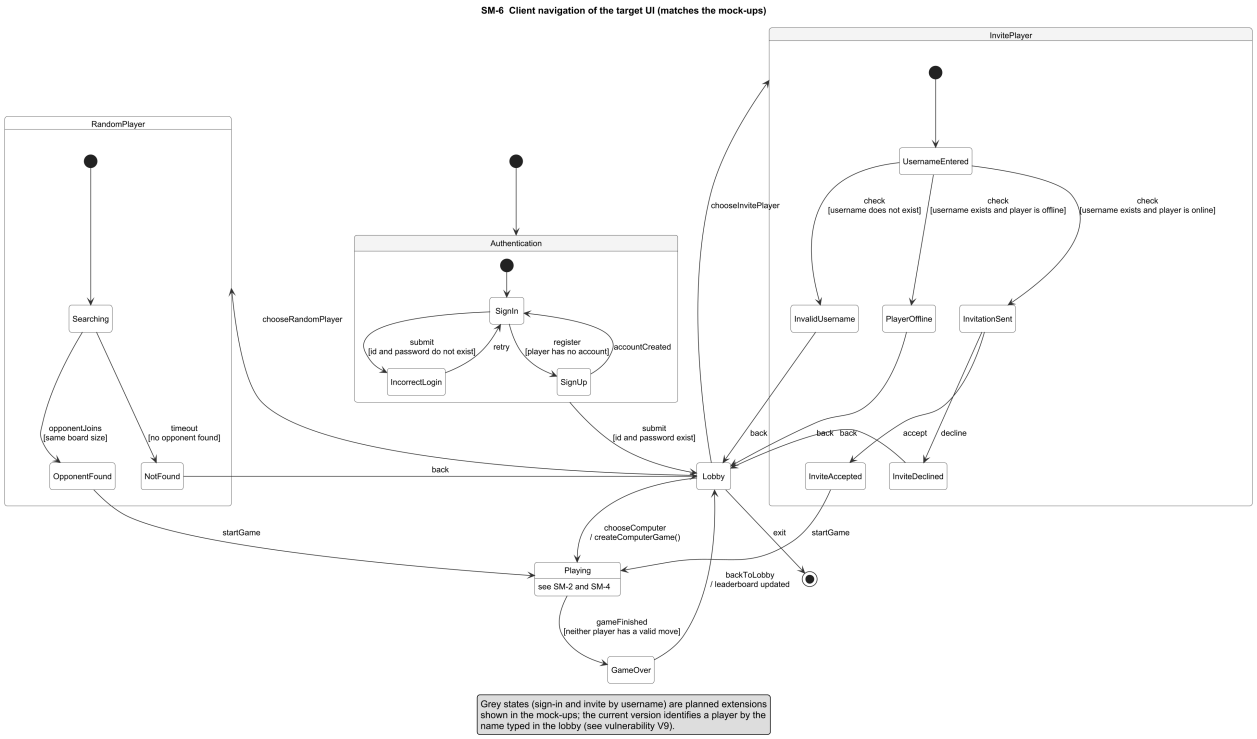


Figure 13: SM-6: Client navigation of the target UI (extra, matches the mock-ups)

7 Sequence Diagrams

The four sequence diagrams use UML 2.x frames (`sd`, `alt`, `opt`, `loop`, `break`, `ref`). There are two main diagrams, each with one sub-sequence diagram referenced by a `ref` fragment:

Diagram	Use case	Shows
SD-1	UC-01 Join Game	Lobby to server, waiting loop with cancel (Ext 1a, 1b)
SD-1a	UC-01 step 2	Pairing, random colours, building the starting board (Ext 2a)
SD-2	UC-02 Make Move	Move from click to saved board; illegal move (Ext 2a)
SD-2a	UC-03 / UC-04	Pass the turn, skip a turn, or end the game

Every message is a real operation from Figure 2 or Figure 3. Notes give the matching use case step or extension.

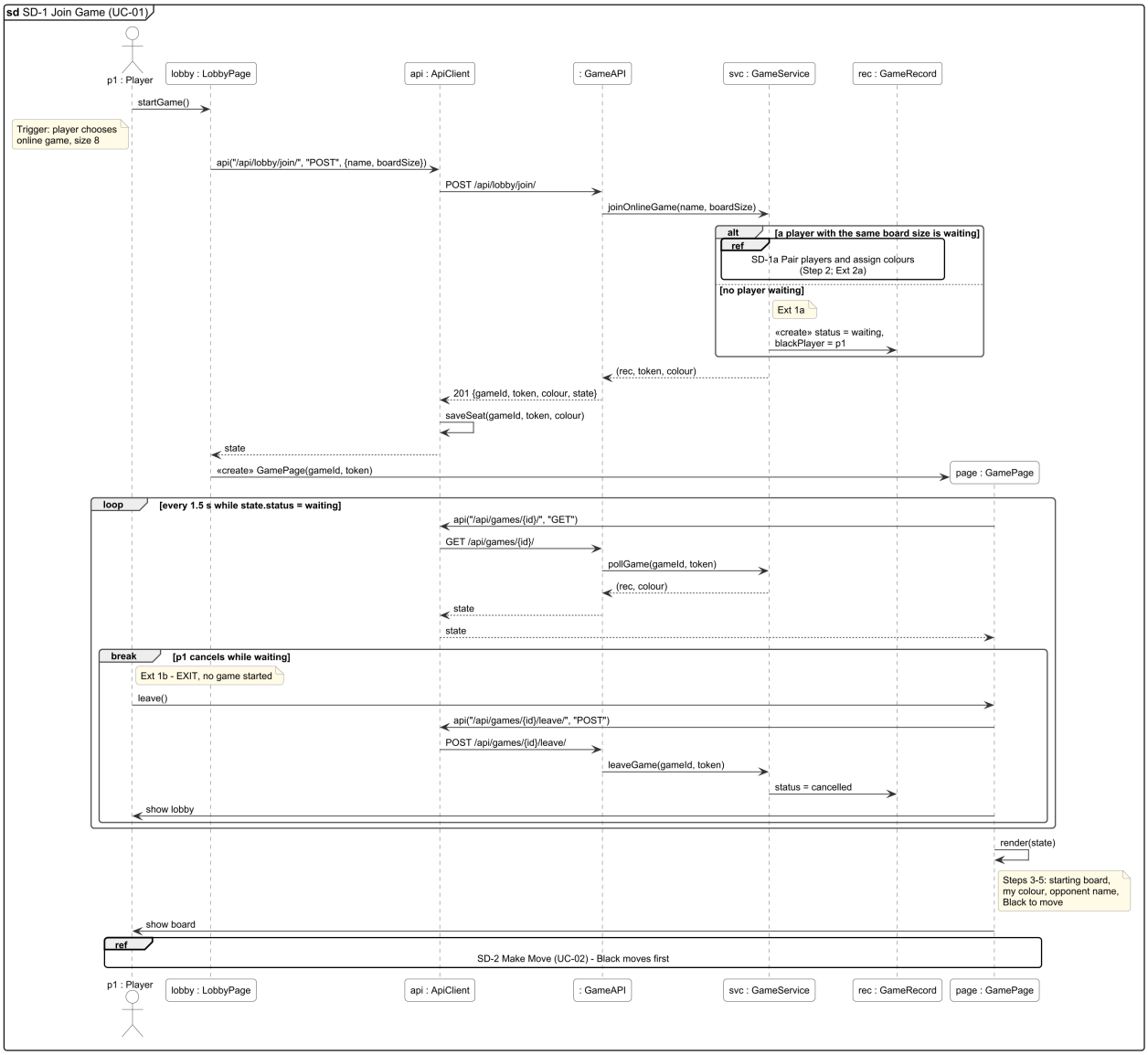


Figure 14: SD-1: Join Game (UC-01)

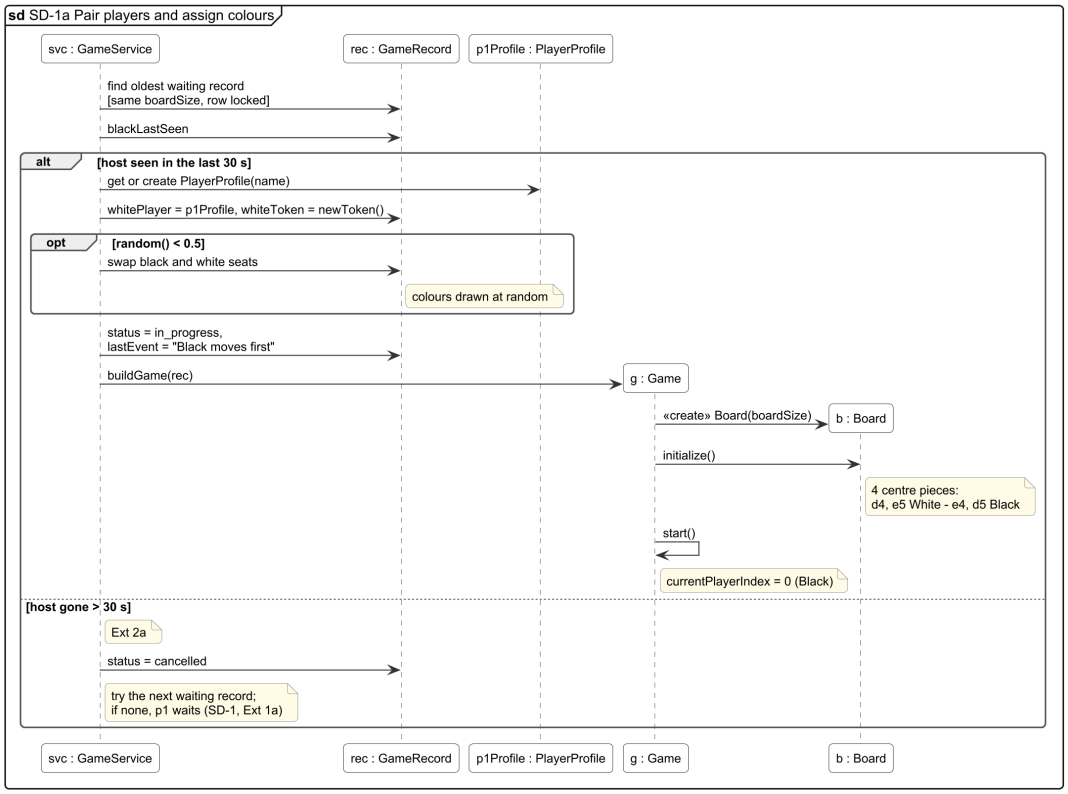


Figure 15: SD-1a: Pair players and assign colours

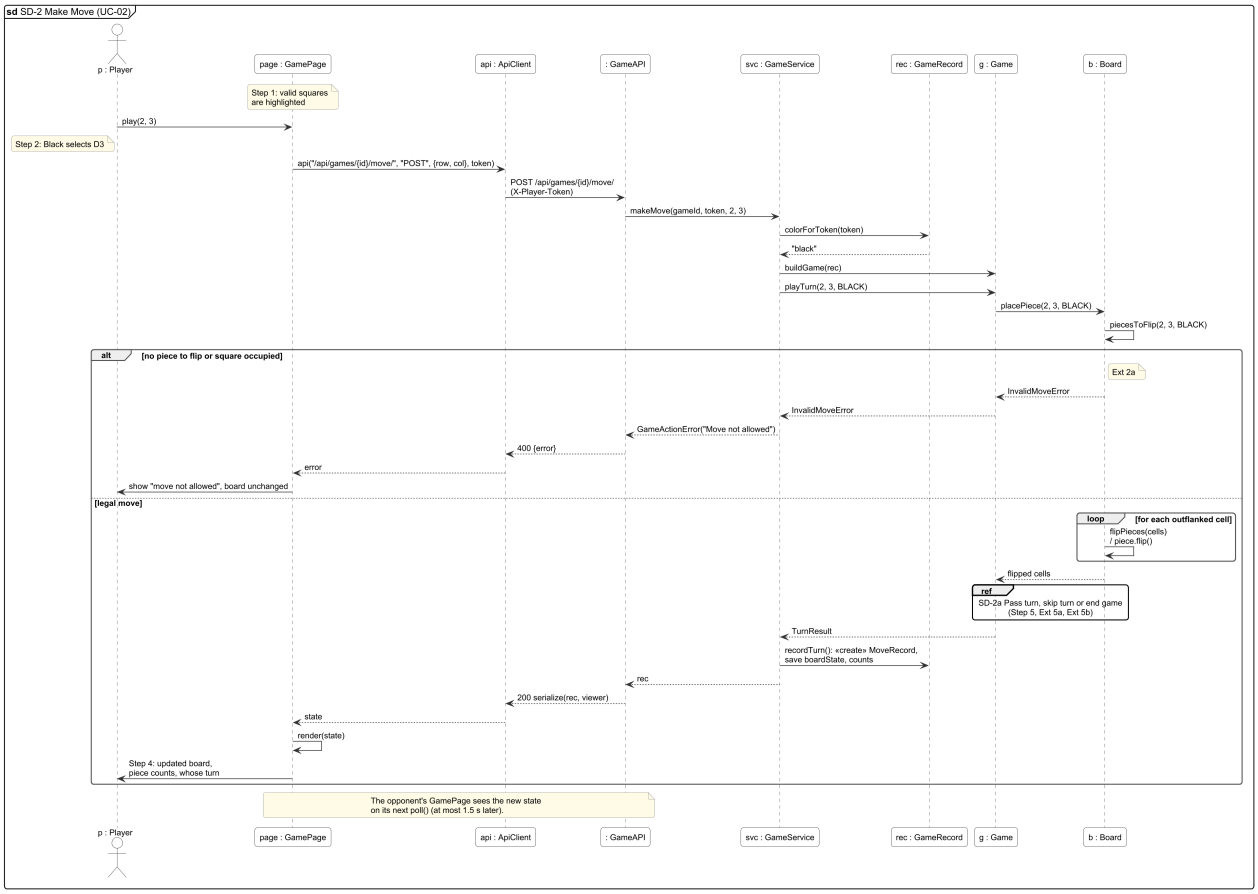


Figure 16: SD-2: Make Move (UC-02)

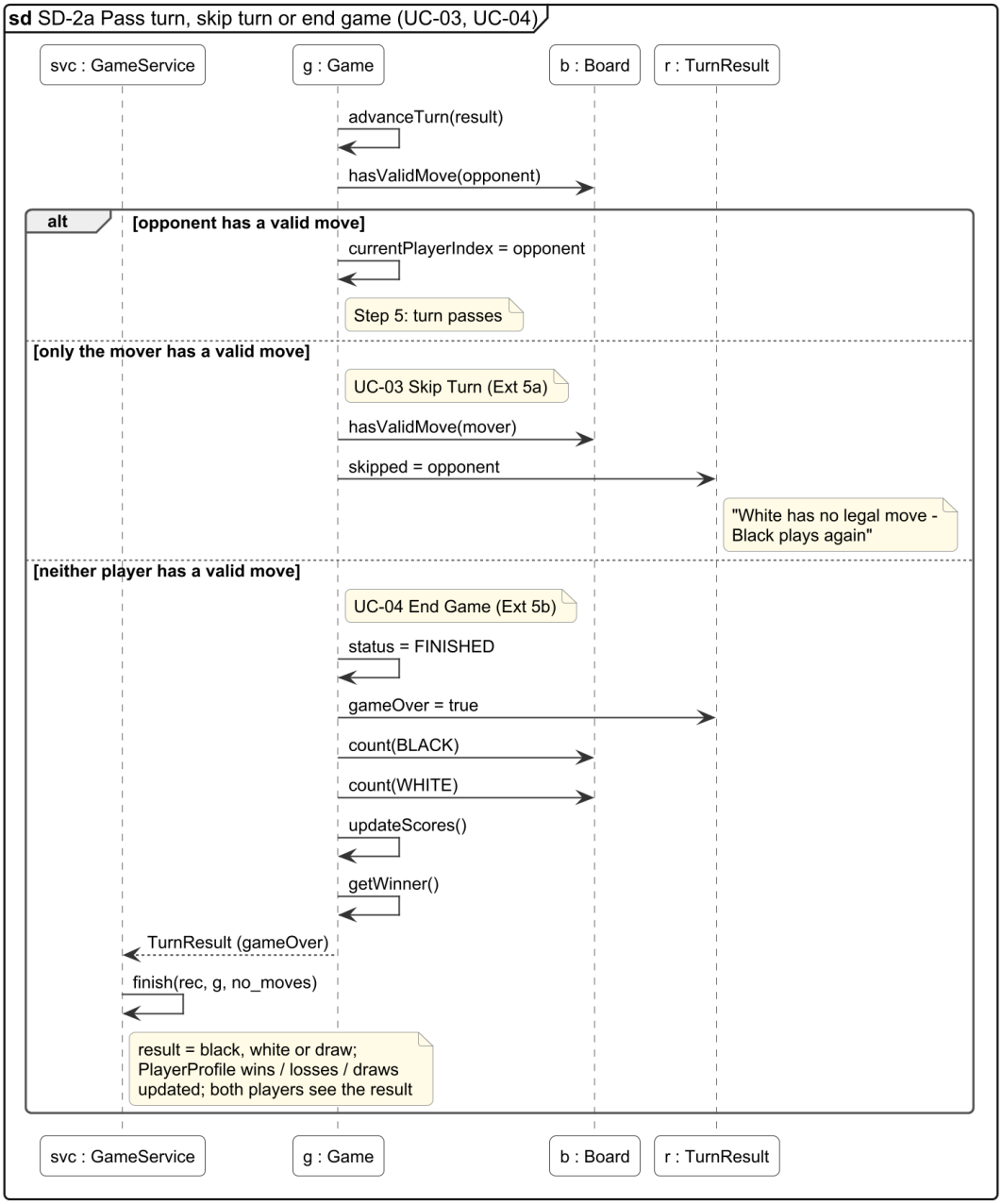


Figure 17: SD-2a: Pass turn, skip turn or end game (UC-03, UC-04)

8 Communication Diagrams

Each communication (collaboration) diagram is drawn from one sequence diagram, with the same objects and messages. Numbers show the nesting of calls (1.1 is called while handling 1), letters show alternative branches, $*$ shows repetition, and guards are in square brackets.

CD-1 Join Game (UC-01) - communication diagram from SD-1

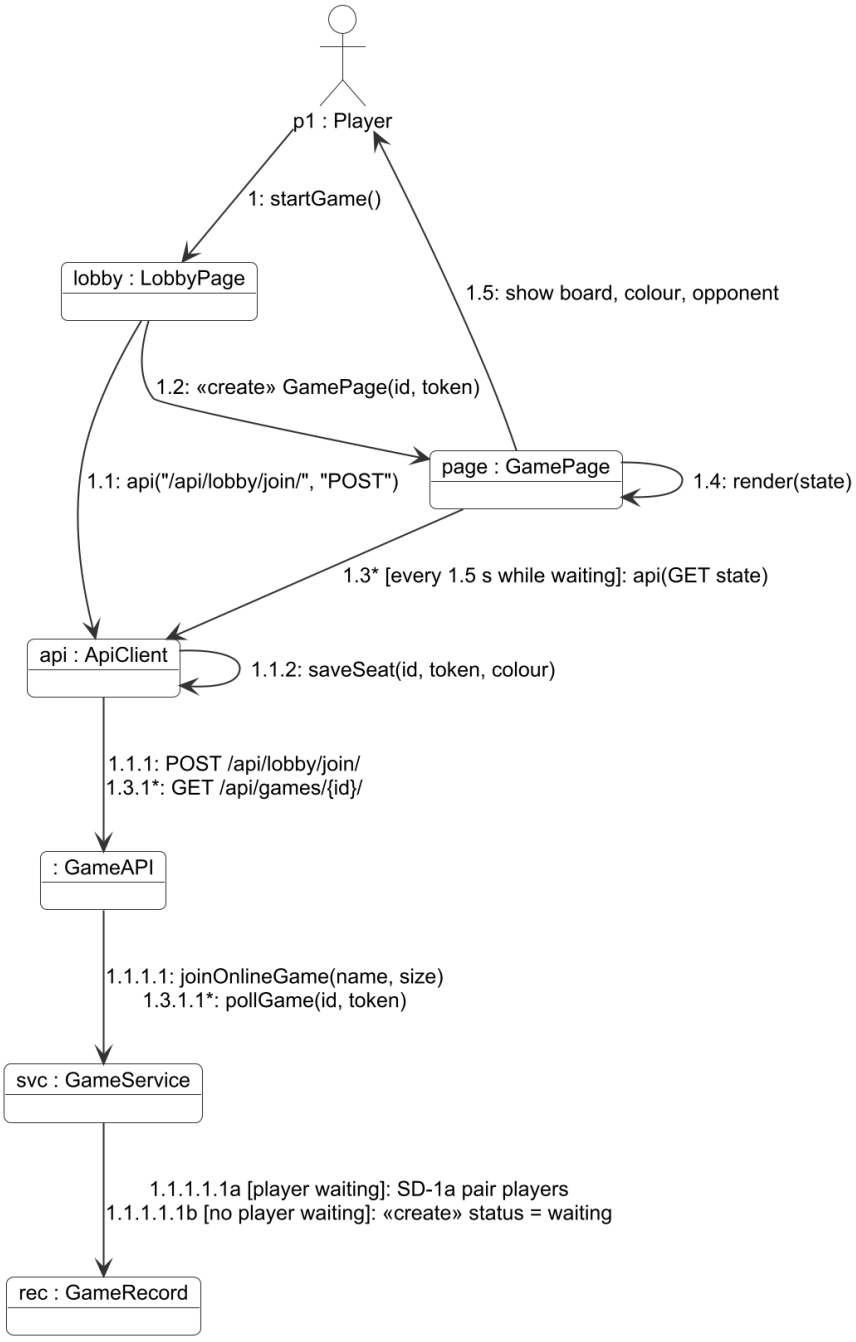


Figure 18: CD-1: Join Game (from SD-1)

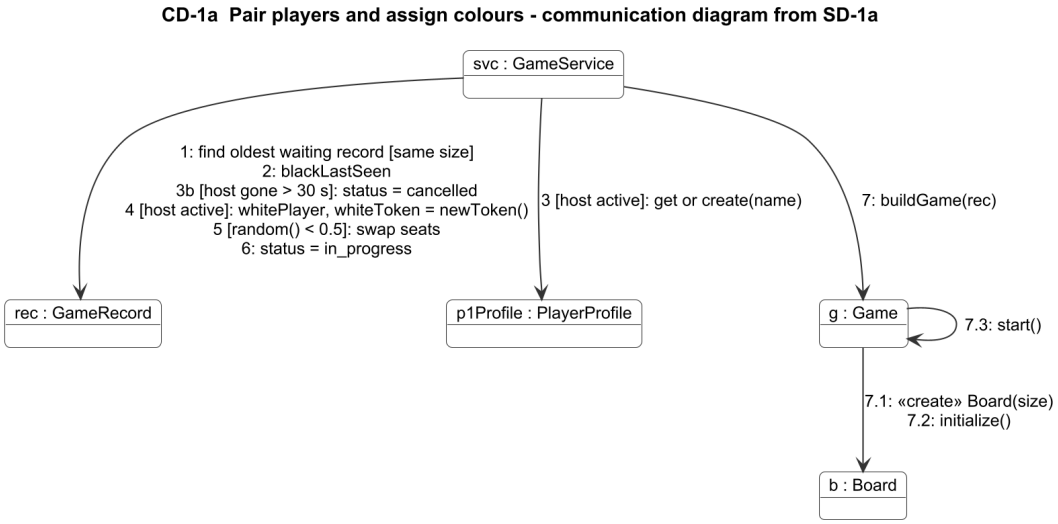


Figure 19: CD-1a: Pair players and assign colours (from SD-1a)

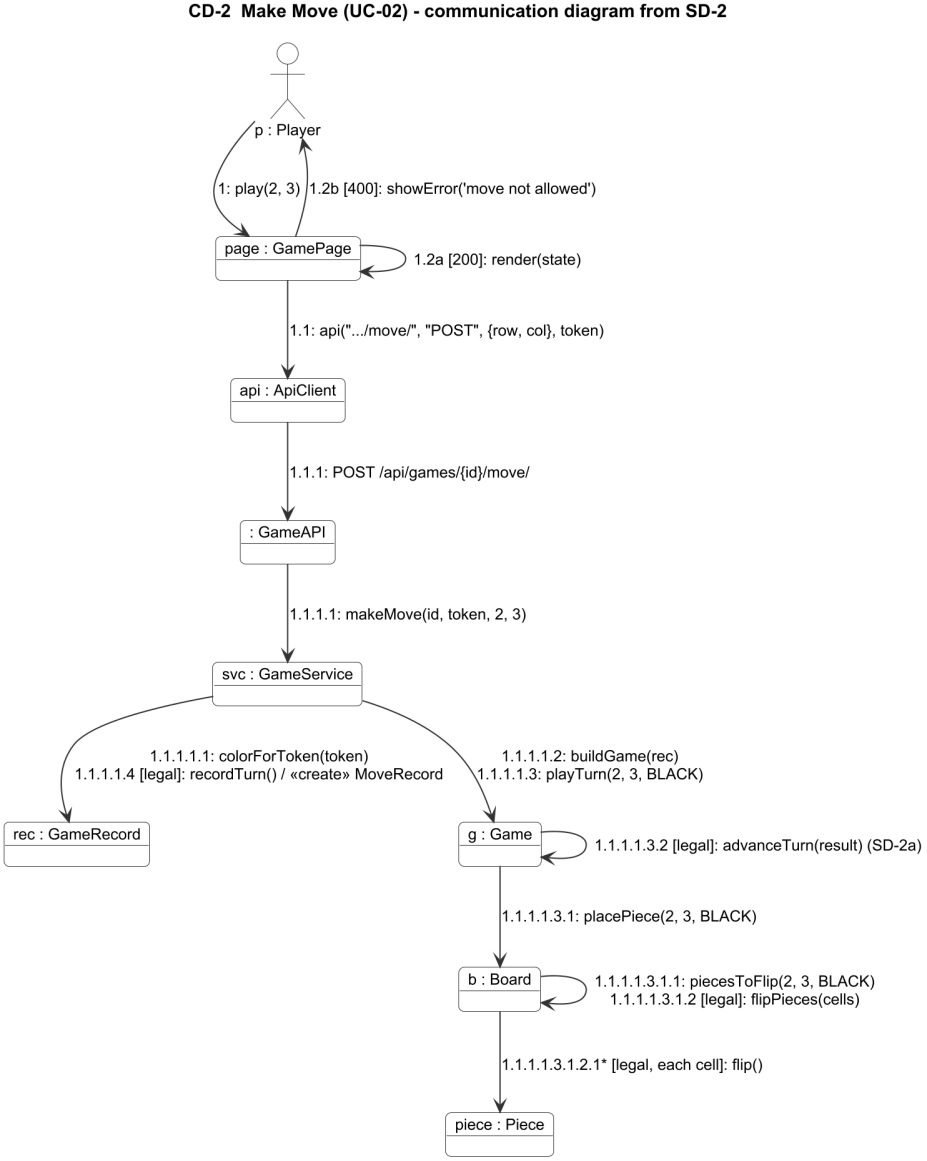


Figure 20: CD-2: Make Move (from SD-2)

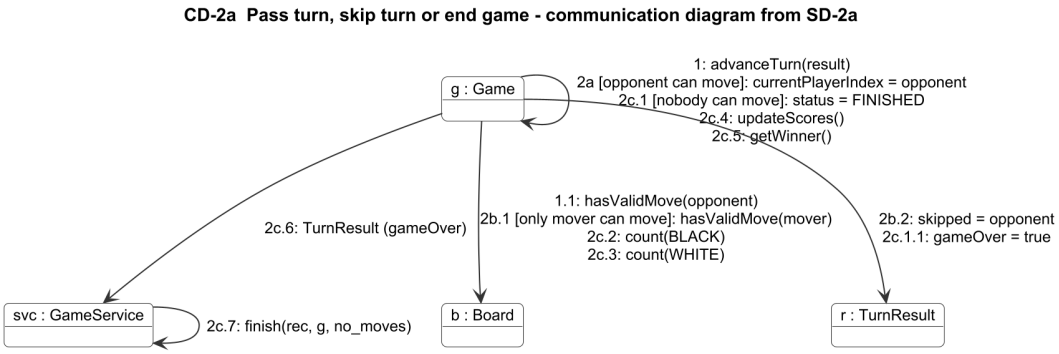


Figure 21: CD-2a: Pass turn, skip turn or end game (from SD-2a)

9 Class Skeletons

The skeletons come from the refined class diagrams and use our implementation languages: Python for the server and JavaScript for the browser. Each class shows its main attributes and methods with a short docstring; bodies are left empty. The full implementation is on GitHub.

9.1 Game rules – `domain.py`

```python
"""MyReversi - class skeletons for the game rules (package: game).

Based on the refined domain class diagram (Fig. 1).
"""
from abc import ABC, abstractmethod
from enum import Enum


class PieceColor(Enum):
    BLACK = "black"
    WHITE = "white"

    def opponent(self):
        """Return the other colour."""
        pass


class GameStatus(Enum):
    WAITING = "waiting"
    IN_PROGRESS = "in_progress"
    FINISHED = "finished"


class InvalidMoveError(Exception):
    """Raised when a move breaks the rules."""


class Piece:
    """A disc on the board."""

    def __init__(self, color):
        self.color = color          # PieceColor

    def flip(self):
        """Change to the opponent's colour."""
        pass
```

```
37
38
39 class Cell:
40     """One square of the board. Holds 0 or 1 Piece."""
41
42     def __init__(self, row, col):
43         self.row = row
44         self.col = col
45         self.piece = None           # Piece or None
46
47     def is_empty(self):
48         pass
49
50
51 class Board:
52     """The grid of cells (6x6, 8x8 or 10x10) and the placement rules."""
53
54     def __init__(self, size=8):
55         self.size = size
56         self.cells = []             # size x size Cells
57
58     def initialize(self):
59         """Place the 4 starting pieces in the centre."""
60         pass
61
62     def is_valid_move(self, row, col, color):
63         """True if the move outflanks at least one opponent piece."""
64         pass
65
66     def get_valid_moves(self, color):
67         """List of (row, col) the player can play (shown to the user)."""
68         pass
69
70     def place_piece(self, row, col, color):
71         """Place a piece and flip the outflanked pieces.
72         Raises InvalidMoveError if the move is not legal."""
73         pass
74
75     def count(self, color):
76         """Number of pieces of this colour (the score)."""
77         pass
78
79
80 class Player(ABC):
81     """A player in the game (human or computer)."""
82
83     def __init__(self, name, color):
84         self.name = name
85         self.color = color          # PieceColor
86         self.score = 0
87
88     @abstractmethod
89     def make_move(self, board):
90         """Return the (row, col) to play."""
91         pass
92
93
94 class HumanPlayer(Player):
95     """A person playing in the browser."""
96
97     def make_move(self, board):
98         pass
99
100
101 class ComputerPlayer(Player):
```

```
    """The computer opponent (UC-05)."""

    def __init__(self, color, difficulty="normal"):
        super().__init__("Computer", color)
        self.difficulty = difficulty  # "easy", "normal" or "hard"

    def make_move(self, board):
        """Choose a move depending on the difficulty."""
        pass


class Game:
    """Controls one game: turns, skipping and the end of the game."""

    def __init__(self, black, white, size=8):
        self.players = [black, white]   # 2 Players, Black first
        self.board = Board(size)
        self.current_player_index = 0   # 0 = Black, 1 = White
        self.status = GameStatus.WAITING

    def start(self):
        """Set up the board. Black plays first (UC-01)."""
        pass

    def play_turn(self, row, col):
        """Play a move for the current player (UC-02),
        then pass the turn, skip (UC-03) or end the game (UC-04)."""
        pass

    def check_game_over(self):
        """True when neither player can move."""
        pass

    def forfeit(self, color):
        """End the game; the other player wins."""
        pass

    def get_winner(self):
        """Return the winning Player, or None for a draw."""
        pass
```

9.2 Server – `server.py`

```
"""MyReversi - class skeletons for the server (packages: api, database).

Based on the refined system class diagram (Fig. 2).
In the real code the three record classes are Django models.
"""


class PlayerProfile:
    """A player's name and results, used for the leaderboard."""

    def __init__(self, name):
        self.name = name
        self.wins = 0
        self.losses = 0
        self.draws = 0


class GameRecord:
    """A saved game, so both players see the same board."""

```

```
    def __init__(self, mode, board_size):
        self.id = None              # unique game id
        self.mode = mode            # "computer" or "online"
        self.board_size = board_size
        self.board_state = []       # the board as a list of rows
        self.status = "waiting"     # waiting, in_progress, finished, cancelled
        self.black_player = None    # PlayerProfile (None = computer)
        self.white_player = None
        self.black_token = ""       # secret token for each player
        self.white_token = ""
        self.result = ""            # "black", "white" or "draw"
        self.moves = []             # list of MoveRecord

    def color_for_token(self, token):
        """Return "black" or "white" for a valid token, else None."""
        pass


class MoveRecord:
    """One move, kept for the game history and replay."""

    def __init__(self, number, color, row, col):
        self.number = number
        self.color = color
        self.row = row
        self.col = col


class GameService:
    """Runs the games on the server (replaces Server from Phase 1)."""

    def create_computer_game(self, name, board_size, difficulty):
        """Start a game against the computer (UC-05)."""
        pass

    def join_online_game(self, name, board_size):
        """Pair with a waiting player, or wait for one (UC-01)."""
        pass

    def make_move(self, game_id, token, row, col):
        """Check the token, play the move and save the game (UC-02)."""
        pass

    def play_computer_turn(self, game_id, token):
        """Let the computer play its move (UC-05)."""
        pass

    def leave_game(self, game_id, token):
        """The player leaves: they forfeit the game."""
        pass

    def poll_game(self, game_id, token):
        """Return the current game state to the client
        (called every 1.5 s)."""
        pass
```

9.3 Browser client – `client.js`

```
// MyReversi - class skeletons for the browser client.
// Based on the refined system class diagram (Fig. 2).

// Sends requests to the server and returns the JSON answer.
class ApiClient {
```

```
  async api(path, method, body, token) {}
}

// Lobby page: the player enters a name and chooses the game type.
class LobbyPage {
  constructor() {
    this.name = "";
    this.mode = "computer";   // "computer" or "online"
    this.boardSize = 8;       // 6, 8 or 10
    this.difficulty = "normal";
  }

  async startGame() {}
}

// Game page: shows the board and sends the player's moves.
class GamePage {
  constructor(gameId, token) {
    this.gameId = gameId;
    this.token = token;       // secret token from the server
    this.state = null;        // last game state received
  }

  async poll() {}             // ask the server for the state every 1.5 s
  async play(row, col) {}     // send a move (UC-02)
  async leave() {}            // leave the game (forfeit)
  render() {}                 // draw the board and the legal moves
  showResult() {}             // show the winner (UC-04)
}
```

10 Technical Vulnerability Review

The Phase 1 feedback asked for a formal review of technical vulnerabilities. We used the STRIDE headings (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) and checked each risk against the classes of Figures 2 and 3.

#	Threat	Example attack	Mitigation	Class / operation	Status
V1	Spoofing	Player B sends a move as Player A	Each seat has a random secret token, returned only to that browser and checked on every action with a constant-time compare	GameRecord.colorForToken, GameSer-vice.makeMove	Mitigated
V2	Tampering	Client sends a fake board or an illegal square	The server is the only authority. The client sends only (row, col); placePiece re-checks the rules and raises InvalidMoveError	Board.placePiece, Game.playTurn	Mitigated
V3	Tampering	Playing out of turn or after the end	NotYourTurnError, GameOverError	Game.playTurn	Mitigated
V4	Tampering (race)	Two moves for the same game at the same moment	Each action runs in one database transaction with a row lock; move numbers are unique per game	GameService, MoveRecord	Mitigated

#	Threat	Example attack	Mitigation	Class / operation	Status
V5	Tampering (bad input)	Non-numeric row, board size 7, 1000-character name	Numbers checked, Board.inBounds, size must be 6/8/10, name 1–30 characters	GameService, Board	Mitigated
V6	Information disclosure / XSS	Name like `<script>` shown to the opponent	Names are inserted as text (textContent / escapeHtml), never as HTML	ApiClient.escapeHtml, GamePage.render	Mitigated
V7	Information disclosure	Token stolen from the browser	Token stored for that game only and useless once it ends; V6 also protects it	ApiClient.saveSeat	Accepted
V8	Information disclosure	Token sniffed on the network	Must be served over HTTPS when deployed	Deployment (Fig. 4)	Open (deploy)
V9	Spoofing	Playing under someone else's name to change their leaderboard	No accounts in this version; a name is not proof of identity. Sign-in is a planned extension (mock-ups)	PlayerProfile	Accepted
V10	Denial of service	Opponent stops responding so the game never ends	30 s reconnection window, then forfeit; stale lobby games are cancelled	GameService.pollGame, joinOnlineGame	Mitigated
V11	Denial of service	Making the computer think forever	Search depth is fixed (3, or 2 on 10×10); the computer only moves when asked by the seated player	ComputerPlayer	Mitigated
V12	Denial of service	Flooding the API with requests	History limited to 50 rows and leaderboard to 20; no rate limiting yet	GameAPI	Open
V13	Elevation of privilege	Debug pages or a known secret key in production	Secret key, debug flag and allowed hosts set by environment variables	settings.py	Open (deploy)
V14	Repudiation	"I never played that move"	Every move is stored with number, colour, square and time	MoveRecord	Mitigated
V15	CSRF	Another site makes the browser send a move	No cookies or sessions; the token must be sent in a custom header that other sites cannot add	GameAPI, ApiClient.api	Mitigated

11 Design Consistency Review

Before submission we did a structured walkthrough of all Phase 2 diagrams against the refined class diagrams, which are the reference for names. Table 5 lists what we found and how it was fixed in this document. The original working files are kept on GitHub for traceability.

Table 5: Inconsistencies found and fixed

Item	Problem found	Fix
Sequence diagrams SD-1, SD-1a	Used classes that are not in the design (LobbyController, Matchmaker) and operations such as startGame(), placeStartingDiscs(), assignColours()	Redrawn with LobbyPage, ApiClient, GameAPI, GameService, GameRecord, Game, Board and their real operations (joinOnlineGame, buildGame, initialize, start). Same fragments kept.
Sequence diagrams SD-2, SD-2a	Planned in the work split but not yet drawn	Drawn for UC-02 and UC-03/UC-04 so that there are four sequence diagrams.
Communication diagrams	Used classes not in the design (ScoreManager, Database) and operations surrender(), validate(), declareWinner(); numbering did not match the sequence diagrams; one label was in French (“Coup invalide”); one file ended with `@endum`	Redrawn from SD-1, SD-1a, SD-2 and SD-2a with the same objects and messages, as the brief asks. Illegal-move and end-of-game cases are kept as guarded branches.
Object diagrams	Attribute names differed from the class diagram (currentTurnIndex, GameState, isObstacle); the computer was Black although UC-05 says the player is Black; cell E5 had col = 4; “IA” instead of AI; scores at the start were 0	Redrawn: currentPlayerIndex, GameStatus, no obstacles, human player is Black, correct 0-based coordinates, starting score 2 each. The mid-game diagram now shows a consistent move number (38 moves for 42 pieces).
State machines	Two sets existed. The first draft had Paused and AwaitingReconnection states, but pause was dropped from the design. The navigation diagram had spelling mistakes and unnamed transitions	The four required diagrams use GameStatus names; every transition is named, with guards. Reconnection is a 30 s guard instead of a state. Cell and navigation diagrams kept as extras, with spelling fixed and all transitions named.
Use cases	Difficulty levels written as easy/medium/hard; online pairing also filtered by difficulty; two variations numbered $1'$	Easy/normal/hard everywhere; online pairing only uses board size (difficulty only applies to computer games); variations renumbered.
Mock-ups vs. design	Sign-in, sign-up and invite screens have no matching classes	Marked as planned extensions in Section 5, UC-01 variation $1''$, SM-6 and V9.
Meeting records	Meeting of 14/10 gave the next meeting as 28/10, but the team met on 21/10	Corrected in Section 12.

12 Team Meeting Records

The team works in short Agile iterations: a weekly meeting (Wednesday, 12:00) to review progress and assign tasks, with the shared Google Doc and GitHub used between meetings.

12.1 Meeting 1 – Wednesday 7 October 2026, 12:00

Attendance	Anna, Ailbhe, Tim, Michel (Blanche and Diane absent)
Agenda	Assign the Phase 2 tasks; front end and back end of the game
Discussion	How to split the game between the browser client and the server
Decisions	Python/Django backend with an HTML/JavaScript client. The backend skeleton was merged on GitHub during the meeting (pull request #2).
Actions	Object diagrams and communication diagrams – Blanche Revised use cases and sequence diagrams – Tim Sequence diagrams – Ailbhe Refined class diagrams and class skeletons – Michel State machine diagrams and walkthrough – Anna UI mock-ups, state machine and meeting notes – Diane
Next meeting	14 October 2026, 12:00

12.2 Meeting 2 – Wednesday 14 October 2026, 12:00

Attendance	Tim, Michel, Anna, Ailbhe, Blanche, Diane (all)
Agenda	Share and discuss individual work so far
Discussion	Progress on each person's part; first UI mock-ups (12/10)
Decisions	Keep the same work split until the deadline
Actions	Same owners as Meeting 1
Next meeting	21 October 2026, 12:00

12.3 Meeting 3 – Wednesday 21 October 2026, 12:00

Attendance	Diane, Blanche, Tim, Michel (Anna and Ailbhe absent)
Agenda	What is left for deliverable 2
Discussion	State machine and sequence diagrams; what went wrong in Phase 1 and how to improve (lecturer feedback)
Decisions	Add the technical vulnerability review asked for in the feedback; put all diagram sources on GitHub (`docs/phase2/`)
Actions	Vulnerability review – Michel Review the scenarios – Ailbhe
Next meeting	28 October 2026, 12:00

12.4 Task board (status on 22 October 2026)

Task	Owner	Status	Date
Revised use cases and use case diagram	Tim	Done	22/10
Sequence diagrams SD-1, SD-1a	Tim	Done	22/10
Sequence diagrams SD-2, SD-2a	Michel (integration)	Done	22/10
Object diagrams	Blanche	Done	21/10
Communication diagrams	Blanche	Done	21/10
Refined class diagrams	Michel	Done	20/10
Class skeletons	Michel	Done	20/10
Technical vulnerability review	Michel	Done	20/10
State machine diagrams	Anna	Done	21/10
Client navigation state machine	Diane	Done	21/10
UI mock-ups	Diane	Done	12/10
Meeting notes	Diane	Done	21/10
Design consistency review and final report	Michel	Done	22/10
Provisional source code (next phase)	All	In progress	13/11

12.5 Activity log (from GitHub commits and Google Doc history)

Date	Who	Activity
Mon 05/10	Ailbhe	Doc: edits to the shared document
Mon 05/10	Anna	GitHub: project structure added
Wed 07/10	Michel, Anna	GitHub: Django backend started; pull request #2 reviewed and merged by Anna; game logic moved into the backend
Wed 07/10	Blanche	Doc: edits to the shared document
Wed 07/10	Michel	GitHub: first playable version (game rules, API, client)
Mon 12/10	Diane	Doc: UI mock-ups
Tue 20/10	Michel	Doc and GitHub: refined class diagrams, class skeletons, vulnerability review
Tue 20/10	Anna	Doc: edits to the shared document
Wed 21/10	Blanche	GitHub and Doc: object diagrams, communication diagrams
Wed 21/10	Diane	GitHub: navigation state machine; Doc: meeting notes
Wed 21/10	Anna	Doc: edits to the shared document
Thu 22/10	Tim	Doc: use cases and sequence diagrams tab
Thu 22/10	Michel	GitHub: Phase 2 branches merged into `main`; consistency review; final report

13 Use of AI

In line with DCU policy, this section records how generative AI was used in Phase 2.

Task	How AI was used
Refined class diagrams and skeletons	An AI assistant (Claude) helped draft the PlantUML for the class diagrams from our Phase 1 diagram and our code, and the first version of the skeletons. The team member checked every class against the code and cut the skeletons down to the main attributes and methods.
Vulnerability review	AI was used to list possible threats under the STRIDE headings. Each threat was then checked against our own design and code, and the status column reflects what is really in the code.
Final report	AI helped compile the team's work into this document, check that names were consistent across diagrams, redraw the inconsistent diagrams in PlantUML and typeset the report in LaTeX. The changes are listed in Section 11.

What AI was not used for. AI did not make design decisions for the team. The game features, the client/server split, the use cases and the decisions on open questions (random colours, no pause, no rematch, 30 s forfeit, difficulty levels) were made by the team in meetings. All AI output was read, checked and corrected by team members, who take responsibility for the content of this report.

A Diagram Source Files

All diagrams in this report were drawn with PlantUML. The text files are in the repository folder `docs/phase2/report/diagrams/` and can be rendered with:

```
java -jar plantuml.jar -tpng -o ../figures *.puml
```

The team's working files (Mermaid object and state diagrams, first communication diagrams) are kept in `docs/phase2/` and in the root of the repository.

`style.iuml`

```
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 12
skinparam ArrowColor #333333
skinparam roundcorner 8
skinparam sequence {
  LifeLineBorderColor #555555
  ParticipantBorderColor #333333
  ParticipantBackgroundColor #FFFFFF
  ActorBorderColor #333333
  ActorBackgroundColor #FFFFFF
  GroupBorderColor #555555
  GroupHeaderFontStyle bold
}
skinparam object {
  BorderColor #333333
  BackgroundColor #FFFFFF
}
skinparam state {
  BorderColor #333333
  BackgroundColor #FFFFFF
}
skinparam note {
  BorderColor #999999
  BackgroundColor #FFFBE6
}
```

```
hide circle
```

use_case_diagram.puml

```
@startuml use_case_diagram
!include style.iuml
left to right direction
skinparam usecase {
  BorderColor #333333
  BackgroundColor #FFFFFF
}
actor Player
actor Opponent
rectangle "MyReversi System" {
  usecase "UC-01\nJoin Game" as UC01
  usecase "UC-05\nPlay Against Computer" as UC05
  usecase "UC-02\nMake Move" as UC02
  usecase "UC-03\nSkip Turn" as UC03
  usecase "UC-04\nEnd Game" as UC04
}
Player -- UC01
Player -- UC05
Player -- UC02
UC01 -- Opponent
UC02 -- Opponent
UC05 ..> UC02 : <<include>>
UC02 <.. UC03 : <<extend>>
UC02 <.. UC04 : <<extend>>
@enduml
```

domain_class_diagram.puml

```
@startuml domain_class_diagram
!pragma layout elk
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 12
skinparam class {
  BackgroundColor #FFFFFF
  BorderColor #333333
  ArrowColor #333333
}
skinparam enum {
  BackgroundColor #F4F4F4
}
title MyReversi - Refined Domain Class Diagram (package game)\nPhase 2 - Group 13

package "game" {

  enum PieceColor <<enumeration>> {
    BLACK
    WHITE
    --
    + opponent() : PieceColor
  }

  enum GameStatus <<enumeration>> {
    WAITING
    IN_PROGRESS
    FINISHED
```

```
}

class Game {
  - players : Player[2] {ordered}
  - board : Board
  - currentPlayerIndex : int = 0
  - status : GameStatus = WAITING
  - forfeitedBy : PieceColor [0..1]
  --
  + Game(black : Player, white : Player, board : Board [0..1], size : int = 8)
  + start() : void
  + resume(currentColor : PieceColor, status : GameStatus) : void
  + getCurrentPlayer() : Player
  + getPlayer(color : PieceColor) : Player
  + validMoves() : Position[*]
  + playTurn(row : int, col : int, color : PieceColor [0..1]) : TurnResult
  + checkGameOver() : boolean
  + forfeit(color : PieceColor) : void
  + getWinner() : Player [0..1]
  - advanceTurn(result : TurnResult) : void
  - updateScores() : void
}

class TurnResult <<value>> {
  + color : PieceColor
  + row : int
  + col : int
  + flipped : Position[*]
  + skipped : PieceColor [0..1]
  + gameOver : boolean = false
}

class Board {
  + {static} SUPPORTED_SIZES : int[3] = {6, 8, 10}
  - size : int {6, 8 or 10}
  - cells : Cell[size][size]
  --
  + Board(size : int = 8)
  + initialize() : void
  + inBounds(row : int, col : int) : boolean
  + getCell(row : int, col : int) : Cell
  + piecesToFlip(row : int, col : int, color : PieceColor) : Cell[*]
  + isValidMove(row : int, col : int, color : PieceColor) : boolean
  + getValidMoves(color : PieceColor) : Position[*]
  + hasValidMove(color : PieceColor) : boolean
  + placePiece(row : int, col : int, color : PieceColor) : Cell[*]
  + flipPieces(cells : Cell[*]) : void
  + count(color : PieceColor) : int
  + isFull() : boolean
  + copy() : Board
  + toList() : String[*][*]
  + {static} fromList(grid : String[*][*]) : Board
}

class Cell {
  - row : int
  - col : int
  - piece : Piece [0..1]
  --
  + Cell(row : int, col : int)
  + setPiece(piece : Piece [0..1]) : void
  + getPiece() : Piece [0..1]
  + isEmpty() : boolean
  + getColor() : PieceColor [0..1]
  + getLabel() : String
```

```
  }

  class Piece {
    - color : PieceColor
    --
    + Piece(color : PieceColor)
    + flip() : void
    + getColor() : PieceColor
  }

  abstract class Player {
    # name : String
    # color : PieceColor
    + score : int = 0
    + isComputer : boolean = false
    --
    + Player(name : String, color : PieceColor)
    + {abstract} makeMove(board : Board) : Position [0..1]
    + updateScore(points : int) : void
    + getName() : String
    + getColor() : PieceColor
  }

  class HumanPlayer {
    - pendingMove : Position [0..1]
    --
    + HumanPlayer(name : String, color : PieceColor)
    + selectSquare(row : int, col : int) : void
    + makeMove(board : Board) : Position [0..1]
  }

  class ComputerPlayer {
    + {static} DIFFICULTIES : String[3] = {easy, normal, hard}
    - difficultyLevel : String
    - rng : Random
    + isComputer : boolean = true
    --
    + ComputerPlayer(color : PieceColor, difficulty : String = "normal")
    + makeMove(board : Board) : Position [0..1]
    - bestGreedyMove(board : Board, moves : Position[*]) : Position
    - calculateBestMove(board : Board, moves : Position[*]) : Position
    - minimax(board : Board, depth : int, alpha : int, beta : int, toMove : PieceColor) : int
    - evaluate(board : Board) : int
  }

  class InvalidMoveError <<exception>>
  class NotYourTurnError <<exception>>
  class GameOverError <<exception>>
}

note "Position = (row : int, col : int)\nrow 0..size-1, col 0..size-1\nshown to players as e.g.
    D3" as N1

' --- associations ------------------------------------------------
Game "1" o-- "2 {ordered}\n+players" Player : is played by >
Game "1" *-- "1\n+board" Board : is played on >
Board "1" *-- "36..100\n+cells {size x size}" Cell : is made of >
Cell "1" *-- "0..1\n+piece" Piece : holds >
Game "1" --> "1\n+status" GameStatus : is in >
Game ..> TurnResult : <<create>>
Piece --> "1\n+color" PieceColor : shows >
Player --> "1\n+color" PieceColor : plays as >

' --- generalisation ----------------------------------------------
Player <|-- HumanPlayer
```

```
Player <|-- ComputerPlayer
InvalidMoveError <|-- NotYourTurnError
InvalidMoveError <|-- GameOverError

' --- dependencies ----------------------------------------------------
Player ..> Board : <<use>> reads
ComputerPlayer ..> Board : <<use>> copies to search
Board ..> InvalidMoveError : <<throw>>
Game ..> NotYourTurnError : <<throw>>
Game ..> GameOverError : <<throw>>
@enduml
```

system_class_diagram.puml

```
@startuml system_class_diagram
!pragma layout elk
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 12
skinparam class {
  BackgroundColor #FFFFFF
  BorderColor #333333
  ArrowColor #333333
}
skinparam package {
  BackgroundColor #FAFAFA
}
title MyReversi - Refined System Class Diagram (client / server)\nPhase 2 - Group 13 - replaces
    Server and Client from Phase 1

package "Client (browser: HTML/CSS + JavaScript)" as client {
  class LobbyPage <<boundary>> {
    - nameInput : String
    - mode : String {computer, online}
    - boardSize : int {6, 8, 10}
    - difficulty : String {easy, normal, hard}
    --
    + startGame() : void
    + resumeLastGame() : void
  }
  class GamePage <<boundary>> {
    - gameId : UUID
    - token : String
    - state : GameState
    - busy : boolean
    --
    + poll() : void
    + play(row : int, col : int) : void
    + leave() : void
    - render(state : GameState) : void
    - renderBoard(board, validMoves) : void
    - showResult(state : GameState) : void
  }
  class HistoryPage <<boundary>> {
    + load(name : String [0..1]) : void
  }
  class ReplayPage <<boundary>> {
    - frames : Frame[*]
    - index : int
    + step(delta : int) : void
  }
  class ApiClient <<utility>> {
```

```
    + api(path : String, method : String, body : Object, token : String) : Object
    + saveSeat(gameId : UUID, token : String, color : String) : void
    + getSeat(gameId : UUID) : Seat [0..1]
    + escapeHtml(text : String) : String
  }
}

package "Server (Python / Django REST framework)" as server {
  class GameAPI <<control>> {
    {method} POST /api/games/computer/
    {method} POST /api/lobby/join/
    {method} GET  /api/games/{id}/
    {method} POST /api/games/{id}/move/
    {method} POST /api/games/{id}/computer-move/
    {method} POST /api/games/{id}/leave/
    {method} GET  /api/games/{id}/replay/
    {method} GET  /api/games/history/
    {method} GET  /api/leaderboard/
  }

  class GameService <<control>> {
    + {static} RECONNECT_WINDOW : Duration = 30 s
    --
    + createComputerGame(name : String, size : int, difficulty : String) : (GameRecord, String,
    String)
    + joinOnlineGame(name : String, size : int) : (GameRecord, String, String)
    + makeMove(gameId : UUID, token : String, row : int, col : int) : GameRecord
    + playComputerTurn(gameId : UUID, token : String) : GameRecord
    + leaveGame(gameId : UUID, token : String) : GameRecord
    + pollGame(gameId : UUID, token : String) : (GameRecord, String)
    + serialize(record : GameRecord, viewer : String) : GameState
    + replayFrames(record : GameRecord) : Frame[*]
    + buildGame(record : GameRecord) : Game
    - finish(record : GameRecord, game : Game, reason : EndReason) : void
    - recordTurn(record : GameRecord, game : Game, result : TurnResult) : void
  }

  class GameActionError <<exception>> {
    + message : String
    + status : int = 400
  }

  package "database (persistence)" {
    class PlayerProfile <<entity>> {
      + name : String {unique, max 30}
      + wins : int = 0
      + losses : int = 0
      + draws : int = 0
      + createdAt : DateTime
      --
      + gamesPlayed() : int
    }
    class GameRecord <<entity>> {
      + id : UUID
      + mode : Mode {computer, online}
      + difficulty : String
      + boardSize : int
      + boardState : JSON
      + currentColor : String
      + status : Status {waiting, in_progress, finished, cancelled}
      + blackToken : String
      + whiteToken : String
      + blackLastSeen : DateTime [0..1]
      + whiteLastSeen : DateTime [0..1]
      + blackCount : int
```

```
      + whiteCount : int
      + result : Result [0..1] {black, white, draw}
      + endReason : EndReason [0..1] {no_moves, forfeit, disconnect}
      + lastEvent : String
      + createdAt : DateTime
      + finishedAt : DateTime [0..1]
      --
      + playerName(color : String) : String
      + colorForToken(token : String) : String [0..1]
    }
    class MoveRecord <<entity>> {
      + number : int {unique per game}
      + color : String
      + row : int
      + col : int
      + flippedCount : int
      + createdAt : DateTime
      --
      + getLabel() : String
    }
  }

  package "game (domain - see domain class diagram)" as domain {
    class Game
    class Board
    abstract class Player
    class TurnResult
  }
}

' --- client side ------------------------------------------------------
LobbyPage ..> ApiClient : <<use>>
GamePage ..> ApiClient : <<use>>
HistoryPage ..> ApiClient : <<use>>
ReplayPage ..> ApiClient : <<use>>
LobbyPage ..> GamePage : opens >

' --- client <-> server (HTTP + JSON, polled every 1.5 s) -----
ApiClient "0..*" --> "1" GameAPI : sends requests to\n(HTTP/JSON, X-Player-Token,\npolled every
    1.5 s) >

' --- server side ------------------------------------------------------
GameAPI ..> GameService : <<use>> delegates to
GameService ..> GameActionError : <<throw>>
GameService ..> Game : <<create>> rebuilds per request
GameService ..> PlayerProfile : <<use>>
GameService ..> GameRecord : <<use>> locks and saves
GameService ..> MoveRecord : <<create>>
GameService ..> TurnResult : reads

GameRecord "0..*" --> "0..1\n+blackPlayer" PlayerProfile : black seat taken by >
GameRecord "0..*" --> "0..1\n+whitePlayer" PlayerProfile : white seat taken by >
GameRecord "1" *-- "0..*\n+moves {ordered by number}" MoveRecord : records >

Game "1" *-- "1" Board : is played on >
Game "1" o-- "2" Player : is played by >
Game ..> TurnResult : <<create>>

note right of GameRecord
  A GameRecord is the saved form of one Game.
  GameService.buildGame() turns it into
  Game + Board + Players at the start of each
  request, and writes the result back after.
  A null player slot = the computer.
end note
```

```
@enduml
```

deployment_diagram.puml

```
@startuml deployment_diagram
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 12
title MyReversi - Deployment Diagram (section 3.2.1 client/server)\nPhase 2 - Group 13

node "Player 1 device" as pc1 <<device>> {
  node "Web browser" as b1 <<execution environment>> {
    artifact "index.html / game.html\nhistory.html / replay.html" as ui1
    artifact "common.js, game.js ..." as js1
  }
}

node "Player 2 device\n(online games only)" as pc2 <<device>> {
  node "Web browser" as b2 <<execution environment>> {
    artifact "same client files" as ui2
  }
}

node "MyReversi server" as srv <<device>> {
  node "Django 6.1 + Django REST framework" as dj <<execution environment>> {
    component "api (GameAPI, GameService)" as api
    component "game (domain rules)" as game
    component "database (models)" as dbapp
  }
  database "SQLite\n(db.sqlite3)" as db
}

pc1 -- srv : HTTP(S) / JSON\npolled every 1.5 s
pc2 -- srv : HTTP(S) / JSON\npolled every 1.5 s
api ..> game
api ..> dbapp
dbapp -- db : Django ORM
@enduml
```

od1_computer_game_start.puml

```
@startuml od1_computer_game_start
!include style.iuml
title OD-1  Start of a game against the computer (8 x 8, after Game.start())
object "game1 : Game" as game1 {
  currentPlayerIndex = 0
  status = IN_PROGRESS
}
object "board1 : Board" as board1 {
  size = 8
}
object "rose : HumanPlayer" as rose {
  name = "Rose"
  color = BLACK
  score = 2
  isComputer = false
}
object "cpu : ComputerPlayer" as cpu {
  name = "Computer"
  color = WHITE
  score = 2
```

```
  isComputer = true
  difficultyLevel = "normal"
}
object "d4 : Cell" as d4 {
  row = 3
  col = 3
}
object "e4 : Cell" as e4 {
  row = 3
  col = 4
}
object "d5 : Cell" as d5 {
  row = 4
  col = 3
}
object "e5 : Cell" as e5 {
  row = 4
  col = 4
}
object "p1 : Piece" as p1 {
  color = WHITE
}
object "p2 : Piece" as p2 {
  color = BLACK
}
object "p3 : Piece" as p3 {
  color = BLACK
}
object "p4 : Piece" as p4 {
  color = WHITE
}
game1 --> board1 : board
game1 --> rose : players[0]
game1 --> cpu : players[1]
board1 --> d4 : cells
board1 --> e4 : cells
board1 --> d5 : cells
board1 --> e5 : cells
d4 --> p1 : piece
e4 --> p2 : piece
d5 --> p3 : piece
e5 --> p4 : piece
note bottom of board1
  The other 60 cells exist but are empty
  (piece = null). Rows/columns are 0-based:
  d4 = row 3, col 3. Black (Rose) moves first.
end note
@enduml
```

od2_online_game_midway.puml

```
@startuml od2_online_game_midway
!include style.iuml
title OD-2  Online game in progress (after move 38, White to move)
object "record7 : GameRecord" as rec {
  mode = online
  boardSize = 8
  status = in_progress
  currentColor = "white"
  blackCount = 24
  whiteCount = 18
}
object "jeanneProfile : PlayerProfile" as jp {
```

```
  name = "Jeanne"
  wins = 3
  losses = 1
}
object "nicolasProfile : PlayerProfile" as np {
  name = "Nicolas"
  wins = 0
  losses = 2
}
object "move38 : MoveRecord" as m38 {
  number = 38
  color = "black"
  row = 4
  col = 5
  flippedCount = 3
}
object "game2 : Game" as game2 {
  currentPlayerIndex = 1
  status = IN_PROGRESS
}
object "board2 : Board" as board2 {
  size = 8
}
object "jeanne : HumanPlayer" as jeanne {
  name = "Jeanne"
  color = BLACK
  score = 24
}
object "nicolas : HumanPlayer" as nicolas {
  name = "Nicolas"
  color = WHITE
  score = 18
}
object "f5 : Cell" as f5 {
  row = 4
  col = 5
}
object "e5 : Cell" as e5 {
  row = 4
  col = 4
}
object "q1 : Piece" as q1 {
  color = BLACK
}
object "q2 : Piece" as q2 {
  color = BLACK
}
rec --> jp : blackPlayer
rec --> np : whitePlayer
rec *-- m38 : moves[37]
game2 --> board2 : board
game2 --> jeanne : players[0]
game2 --> nicolas : players[1]
board2 --> f5 : cells
board2 --> e5 : cells
f5 --> q1 : piece
e5 --> q2 : piece
rec ..> game2 : GameService.buildGame()
note right of e5
  e5 was White and was flipped
  to Black by move 38 (F5).
end note
@enduml
```

sm1_game.puml

```
@startuml sm1_game
!include style.iuml
title SM-1  Game (GameStatus) - state diagram
hide empty description
[*] --> WAITING : joinOnlineGame()\n[no opponent waiting]\n/ GameRecord saved
[*] --> IN_PROGRESS : joinOnlineGame()\n[opponent already waiting]\n/ colours drawn at random
[*] --> IN_PROGRESS : createComputerGame()\n/ player is Black
WAITING --> IN_PROGRESS : secondPlayerJoins\n[same board size]\n/ colours drawn at random
WAITING --> CANCELLED : leaveGame()\n[still waiting]
WAITING --> CANCELLED : pollTimeout\n[host gone > 30 s]
IN_PROGRESS --> FINISHED : playTurn()\n[neither player has a valid move]\n/ result = more pieces
     or draw
IN_PROGRESS --> FINISHED : leaveGame() or pollGame()\n[player quits or opponent gone > 30 s]\n/
    forfeit(color)
FINISHED --> [*]
CANCELLED --> [*]
FINISHED : entry / count pieces, set winner or draw
FINISHED : entry / update PlayerProfile wins, losses, draws
@enduml
```

sm2_in_progress.puml

```
@startuml sm2_in_progress
!include style.iuml
title SM-2  Game: IN_PROGRESS - sub-state diagram (UC-02, UC-03, UC-04)
hide empty description
state IN_PROGRESS {
  [*] --> BlackToMove : start()\n/ Black always moves first
  BlackToMove --> WhiteToMove : playTurn(row, col)\n[legal move and White has a valid move]
  WhiteToMove --> BlackToMove : playTurn(row, col)\n[legal move and Black has a valid move]
  BlackToMove --> BlackToMove : playTurn(row, col)\n[legal move and only Black has a valid move]\
    n/ White skipped
  WhiteToMove --> WhiteToMove : playTurn(row, col)\n[legal move and only White has a valid move]\
    n/ Black skipped
  BlackToMove --> BlackToMove : playTurn(row, col)\n[illegal move]\n/ InvalidMoveError, board
    unchanged
  WhiteToMove --> WhiteToMove : playTurn(row, col)\n[illegal move]\n/ InvalidMoveError, board
    unchanged
  BlackToMove --> [*] : playTurn(row, col)\n[neither player has a valid move]
  WhiteToMove --> [*] : playTurn(row, col)\n[neither player has a valid move]
}
IN_PROGRESS --> FINISHED : gameOver\n[neither player has a valid move]
IN_PROGRESS --> FINISHED : forfeit(color)\n[player quits or opponent gone > 30 s]
note bottom of IN_PROGRESS
  Each legal move runs Board.placePiece() -> Board.flipPieces()
  -> Piece.flip(), then GameService saves a MoveRecord.
  Against the computer, playComputerTurn() calls
  ComputerPlayer.makeMove(board) and then Game.playTurn().
end note
@enduml
```

sm3_game_page.puml

```
@startuml sm3_game_page
!include style.iuml
title SM-3  GamePage (browser client) - state diagram
hide empty description
[*] --> Lobby
Lobby --> WaitingForOpponent : startGame()\n[mode = online and status = WAITING]
```

```
Lobby --> Playing : startGame()\n[status = IN_PROGRESS]\n(paired at once, or computer game)
WaitingForOpponent --> WaitingForOpponent : poll() every 1.5 s\n[status = WAITING]
WaitingForOpponent --> Playing : poll()\n[status = IN_PROGRESS]
WaitingForOpponent --> Lobby : leave()\n[player cancels]
Playing --> ShowingResult : poll() or play()\n[status = FINISHED]\n/ showResult()
Playing --> Lobby : leave()\n[player confirms quit]\n/ game forfeited
ShowingResult --> Lobby : backToLobby()\n[player clicks "Lobby"]
ShowingResult --> [*] : closePage()
Lobby --> [*] : closePage()
@enduml
```

sm4_playing.puml

```
@startuml sm4_playing
!include style.iuml
title SM-4  GamePage: Playing - sub-state diagram
hide empty description
state Playing {
  [*] --> MyTurn : enterGame\n[my colour = current colour]
  [*] --> OpponentTurn : enterGame\n[my colour <> current colour]
  MyTurn --> SendingMove : play(row, col)\n[square is highlighted]\n/ POST /api/games/{id}/move/
  SendingMove --> OpponentTurn : response 200\n[opponent has a valid move]\n/ render()
  SendingMove --> MyTurn : response 200\n[only I have a valid move]\n/ render(), show "opponent
    skipped"
  SendingMove --> MyTurn : response 400\n[move rejected]\n/ show error, board unchanged
  OpponentTurn --> MyTurn : poll() every 1.5 s\n[human opponent has moved]\n/ render()
  OpponentTurn --> MyTurn : computer-move response\n[computer opponent has moved]\n/ render()
  OpponentTurn --> OpponentTurn : poll()\n[only the opponent has a valid move]\n/ show "I am
    skipped"
  MyTurn --> [*] : poll()\n[status = FINISHED]
  OpponentTurn --> [*] : poll()\n[status = FINISHED]
  SendingMove --> [*] : response 200\n[status = FINISHED]
}
Playing --> ShowingResult : gameFinished\n[status = FINISHED]
@enduml
```

sm5_cell.puml

```
@startuml sm5_cell
!include style.iuml
title SM-5  Cell - state diagram
hide empty description
[*] --> Empty : Board.initialize()\n[not a centre square]
[*] --> Occupied : Board.initialize()\n[centre square]\n/ setPiece(starting piece)
Empty --> Occupied : placePiece(row, col, color)\n[legal move]\n/ setPiece(new Piece(color))
state Occupied {
  [*] --> Black : [piece.color = BLACK]
  [*] --> White : [piece.color = WHITE]
  Black --> White : flip()\n[outflanked by White]
  White --> Black : flip()\n[outflanked by Black]
}
Empty : isEmpty() = true
Occupied : isEmpty() = false
note right of Occupied
  A piece is never removed once
  placed: it can only change colour.
end note
@enduml
```

sm6_client_navigation.puml

```
@startuml sm6_client_navigation
!include style.iuml
title SM-6  Client navigation of the target UI (matches the mock-ups)
hide empty description
state Authentication #F4F4F4 {
  [*] --> SignIn
  SignIn --> IncorrectLogin : submit\n[id and password do not exist]
  IncorrectLogin --> SignIn : retry
  SignIn --> SignUp : register\n[player has no account]
  SignUp --> SignIn : accountCreated
}
[*] --> Authentication
Authentication --> Lobby : submit\n[id and password exist]
state RandomPlayer {
  [*] --> Searching
  Searching --> OpponentFound : opponentJoins\n[same board size]
  Searching --> NotFound : timeout\n[no opponent found]
}
Lobby --> RandomPlayer : chooseRandomPlayer
NotFound --> Lobby : back
OpponentFound --> Playing : startGame
state InvitePlayer #F4F4F4 {
  [*] --> UsernameEntered
  UsernameEntered --> InvalidUsername : check\n[username does not exist]
  UsernameEntered --> InvitationSent : check\n[username exists and player is online]
  UsernameEntered --> PlayerOffline : check\n[username exists and player is offline]
  InvitationSent --> InviteAccepted : accept
  InvitationSent --> InviteDeclined : decline
}
Lobby --> InvitePlayer : chooseInvitePlayer
InvalidUsername --> Lobby : back
PlayerOffline --> Lobby : back
InviteDeclined --> Lobby : back
InviteAccepted --> Playing : startGame
Lobby --> Playing : chooseComputer\n/ createComputerGame()
Playing : see SM-2 and SM-4
Playing --> GameOver : gameFinished\n[neither player has a valid move]
GameOver --> Lobby : backToLobby\n/ leaderboard updated
Lobby --> [*] : exit
legend bottom
  Grey states (sign-in and invite by username) are planned extensions
  shown in the mock-ups; the current version identifies a player by the
  name typed in the lobby (see vulnerability V9).
endlegend
@enduml
```

sd1_join_game.puml

```
@startuml sd1_join_game
!include style.iuml
mainframe **sd** SD-1 Join Game (UC-01)
autonumber stop
actor "p1 : Player" as P1
participant "lobby : LobbyPage" as L
participant "api : ApiClient" as A
participant ": GameAPI" as API
participant "svc : GameService" as S
participant "rec : GameRecord" as R

P1 -> L : startGame()
note left of P1 : Trigger: player chooses\nonline game, size 8
```

```
L -> A : api("/api/lobby/join/", "POST", {name, boardSize})
A -> API : POST /api/lobby/join/
API -> S : joinOnlineGame(name, boardSize)
alt a player with the same board size is waiting
  ref over S, R : SD-1a Pair players and assign colours\n(Step 2; Ext 2a)
else no player waiting
  note right of S : Ext 1a
  S -> R : <<create>> status = waiting,\nblackPlayer = p1
end
S --> API : (rec, token, colour)
API --> A : 201 {gameId, token, colour, state}
A -> A : saveSeat(gameId, token, colour)
A --> L : state
create participant "page : GamePage" as G
L -> G : <<create>> GamePage(gameId, token)
loop every 1.5 s while state.status = waiting
  G -> A : api("/api/games/{id}/", "GET")
  A -> API : GET /api/games/{id}/
  API -> S : pollGame(gameId, token)
  S --> API : (rec, colour)
  API --> A : state
  A --> G : state
  break p1 cancels while waiting
    note right of P1 : Ext 1b - EXIT, no game started
    P1 -> G : leave()
    G -> A : api("/api/games/{id}/leave/", "POST")
    A -> API : POST /api/games/{id}/leave/
    API -> S : leaveGame(gameId, token)
    S -> R : status = cancelled
    G -> P1 : show lobby
  end
end
G -> G : render(state)
note right of G : Steps 3-5: starting board,\nmy colour, opponent name,\nBlack to move
G -> P1 : show board
ref over P1, G : SD-2 Make Move (UC-02) - Black moves first
@enduml
```

sd1a_pair_players.puml

```
@startuml sd1a_pair_players
!include style.iuml
mainframe **sd** SD-1a Pair players and assign colours
participant "svc : GameService" as S
participant "rec : GameRecord" as R
participant "p1Profile : PlayerProfile" as P

S -> R : find oldest waiting record\n[same boardSize, row locked]
S -> R : blackLastSeen
alt host seen in the last 30 s
  S -> P : get or create PlayerProfile(name)
  S -> R : whitePlayer = p1Profile, whiteToken = newToken()
  opt random() < 0.5
    S -> R : swap black and white seats
    note right of R : colours drawn at random
  end
  S -> R : status = in_progress,\nlastEvent = "Black moves first"
  create participant "g : Game" as G
  S -> G : buildGame(rec)
  create participant "b : Board" as B
  G -> B : <<create>> Board(boardSize)
  G -> B : initialize()
  note right of B : 4 centre pieces:\nd4, e5 White - e4, d5 Black
```

```
  G -> G : start()
  note right of G : currentPlayerIndex = 0 (Black)
else host gone > 30 s
  note right of S : Ext 2a
  S -> R : status = cancelled
  note right of S : try the next waiting record;\nif none, p1 waits (SD-1, Ext 1a)
end
@enduml
```

sd2_make_move.puml

```
@startuml sd2_make_move
!include style.iuml
mainframe **sd** SD-2 Make Move (UC-02)
actor "p : Player" as P
participant "page : GamePage" as G
participant "api : ApiClient" as A
participant ": GameAPI" as API
participant "svc : GameService" as S
participant "rec : GameRecord" as R
participant "g : Game" as GM
participant "b : Board" as B

note over G : Step 1: valid squares\nare highlighted
P -> G : play(2, 3)
note left of P : Step 2: Black selects D3
G -> A : api("/api/games/{id}/move/", "POST", {row, col}, token)
A -> API : POST /api/games/{id}/move/\n(X-Player-Token)
API -> S : makeMove(gameId, token, 2, 3)
S -> R : colorForToken(token)
R --> S : "black"
S -> GM : buildGame(rec)
S -> GM : playTurn(2, 3, BLACK)
GM -> B : placePiece(2, 3, BLACK)
B -> B : piecesToFlip(2, 3, BLACK)
alt no piece to flip or square occupied
  note right of B : Ext 2a
  B --> GM : InvalidMoveError
  GM --> S : InvalidMoveError
  S --> API : GameActionError("Move not allowed")
  API --> A : 400 {error}
  A --> G : error
  G -> P : show "move not allowed", board unchanged
else legal move
  loop for each outflanked cell
    B -> B : flipPieces(cells)\n/ piece.flip()
  end
  B --> GM : flipped cells
  ref over GM, B : SD-2a Pass turn, skip turn or end game\n(Step 5, Ext 5a, Ext 5b)
  GM --> S : TurnResult
  S -> R : recordTurn(): <<create>> MoveRecord,\nsave boardState, counts
  S --> API : rec
  API --> A : 200 serialize(rec, viewer)
  A --> G : state
  G -> G : render(state)
  G -> P : Step 4: updated board,\npiece counts, whose turn
end
note over G, API : The opponent's GamePage sees the new state\non its next poll() (at most 1.5 s
      later).
@enduml
```

sd2a_turn_or_end.puml

```
@startuml sd2a_turn_or_end
!include style.iuml
mainframe **sd** SD-2a Pass turn, skip turn or end game (UC-03, UC-04)
participant "svc : GameService" as S
participant "g : Game" as GM
participant "b : Board" as B
participant "r : TurnResult" as T

GM -> GM : advanceTurn(result)
GM -> B : hasValidMove(opponent)
alt opponent has a valid move
  GM -> GM : currentPlayerIndex = opponent
  note right of GM : Step 5: turn passes
else only the mover has a valid move
  note right of GM : UC-03 Skip Turn (Ext 5a)
  GM -> B : hasValidMove(mover)
  GM -> T : skipped = opponent
  note right of T : "White has no legal move -\nBlack plays again"
else neither player has a valid move
  note right of GM : UC-04 End Game (Ext 5b)
  GM -> GM : status = FINISHED
  GM -> T : gameOver = true
  GM -> B : count(BLACK)
  GM -> B : count(WHITE)
  GM -> GM : updateScores()
  GM -> GM : getWinner()
  GM --> S : TurnResult (gameOver)
  S -> S : finish(rec, g, no_moves)
  note right of S : result = black, white or draw;\nPlayerProfile wins / losses / draws\nupdated;
     both players see the result
end
@enduml
```

cd1_join_game.puml

```
@startuml cd1_join_game
!include style.iuml
allowmixing
title CD-1  Join Game (UC-01) - communication diagram from SD-1
actor "p1 : Player" as P1
object "lobby : LobbyPage" as L
object "api : ApiClient" as A
object ": GameAPI" as API
object "svc : GameService" as S
object "rec : GameRecord" as R
object "page : GamePage" as G
P1 --> L : 1: startGame()
L --> A : 1.1: api("/api/lobby/join/", "POST")
A --> API : 1.1.1: POST /api/lobby/join/\n1.3.1*: GET /api/games/{id}/
API --> S : 1.1.1.1: joinOnlineGame(name, size)\n1.3.1.1*: pollGame(id, token)
S --> R : 1.1.1.1.1a [player waiting]: SD-1a pair players\n1.1.1.1.1b [no player waiting]: <<
    create>> status = waiting
A --> A : 1.1.2: saveSeat(id, token, colour)
L --> G : 1.2: <<create>> GamePage(id, token)
G --> A : 1.3* [every 1.5 s while waiting]: api(GET state)
G --> G : 1.4: render(state)
G --> P1 : 1.5: show board, colour, opponent
@enduml
```

cd1a_pair_players.puml

```
@startuml cd1a_pair_players
!include style.iuml
title CD-1a  Pair players and assign colours - communication diagram from SD-1a
object "svc : GameService" as S
object "rec : GameRecord" as R
object "p1Profile : PlayerProfile" as P
object "g : Game" as G
object "b : Board" as B
S --> R : 1: find oldest waiting record [same size]\n2: blackLastSeen\n3b [host gone > 30 s]: status = cancelled\n4 [host active]: whitePlayer, whiteToken = newToken()\n5 [random() < 0.5]: swap seats\n6: status = in_progress
S --> P : 3 [host active]: get or create(name)
S --> G : 7: buildGame(rec)
G --> B : 7.1: <<create>> Board(size)\n7.2: initialize()
G --> G : 7.3: start()
@enduml
```

cd2_make_move.puml

```
@startuml cd2_make_move
!include style.iuml
allowmixing
title CD-2  Make Move (UC-02) - communication diagram from SD-2
actor "p : Player" as P
object "page : GamePage" as G
object "api : ApiClient" as A
object ": GameAPI" as API
object "svc : GameService" as S
object "rec : GameRecord" as R
object "g : Game" as GM
object "b : Board" as B
object "piece : Piece" as PC
P --> G : 1: play(2, 3)
G --> A : 1.1: api(".../move/", "POST", {row, col}, token)
A --> API : 1.1.1: POST /api/games/{id}/move/
API --> S : 1.1.1.1: makeMove(id, token, 2, 3)
S --> R : 1.1.1.1.1: colorForToken(token)\n1.1.1.1.4 [legal]: recordTurn() / <<create>> MoveRecord
S --> GM : 1.1.1.1.2: buildGame(rec)\n1.1.1.1.3: playTurn(2, 3, BLACK)
GM --> B : 1.1.1.1.3.1: placePiece(2, 3, BLACK)
B --> B : 1.1.1.1.3.1.1: piecesToFlip(2, 3, BLACK)\n1.1.1.1.3.1.2 [legal]: flipPieces(cells)
B --> PC : 1.1.1.1.3.1.2.1* [legal, each cell]: flip()
GM --> GM : 1.1.1.1.3.2 [legal]: advanceTurn(result) (SD-2a)
G --> G : 1.2a [200]: render(state)
G --> P : 1.2b [400]: showError('move not allowed')
@enduml
```

cd2a_turn_or_end.puml

```
@startuml cd2a_turn_or_end
!include style.iuml
title CD-2a  Pass turn, skip turn or end game - communication diagram from SD-2a
object "svc : GameService" as S
object "g : Game" as GM
object "b : Board" as B
object "r : TurnResult" as T
GM --> GM : 1: advanceTurn(result)\n2a [opponent can move]: currentPlayerIndex = opponent\n2c.1 [nobody can move]: status = FINISHED\n2c.4: updateScores()\n2c.5: getWinner()
```

```
GM --> B : 1.1: hasValidMove(opponent)\n2b.1 [only mover can move]: hasValidMove(mover)\n2c.2:
    count(BLACK)\n2c.3: count(WHITE)
GM --> T : 2b.2: skipped = opponent\n2c.1.1: gameOver = true
GM --> S : 2c.6: TurnResult (gameOver)
S --> S : 2c.7: finish(rec, g, no_moves)
@enduml
```